ROBOTICS SOFTWARE STATE OF THE PRACTICE

A STATE-OF-THE-PRACTICE STUDY OF ROBOTICS SOFTWARE FOR MOTION PLANNING AND SIMULATION

BY

ZIYANG FANG, B.Eng.

A Report

submitted to the Department of Computing and Software

and the School of Graduate Studies

of McMaster University

in partial fulfilment of the requirements

for the degree of

Master of Engineering

Master of Engineering (2026) McMaster University

(Department of Computing and Software) Hamilton, Ontario, Canada

TITLE:	A State-of-the-Practice Study of Robotics Software for Motion Planning and Simulation
AUTHOR:	Ziyang Fang B.Eng. (Mechanical Design, Manufacturing and Automation), Fuzhou University, School of Mechanical Engineering, China
SUPERVISOR:	Spencer Smith
NUMBER OF PAGES:	xxii, 149

Lay Abstract

Robots are usually tested in software simulations before they are trusted in the real world. The quality of the software that plans robot motions and simulates robot behaviour therefore matters to everyone who builds or studies robots. This report examines 28 open-source packages for robot motion planning and simulation and grades each one on qualities such as how easy it is to install, how well it is documented, and how actively it is maintained, using only publicly visible evidence. The study finds that this software is generally in good shape: installation support and documentation are strong, although written design information is often missing. The report also shows that a package's popularity does not always match its measured quality.

Abstract

Software for motion planning and simulation (MPS) supports the design, testing, and deployment of robotic systems. Because experiments and applications depend on this software behaving correctly, its engineering quality matters beyond any single project. This report assesses the state of the practice for MPS software by adapting the research-software assessment methodology of Smith et al. Candidate packages were gathered from academic surveys, community-curated lists, and auxiliary LLM-assisted discovery; after consolidation of 295 unique candidate names and filtering by open-source availability, repository-based measurability, and maintenance activity, 28 packages were selected, spanning full simulators, physics and dynamics engines, motion planning libraries, manipulation frameworks, and middleware. Each package was measured in May 2026 against a standard template covering installability, correctness and verifiability, surface reliability, surface robustness, surface usability, maintainability, reusability, surface understandability, and visibility and transparency, together with repository metrics. A commonality analysis describes the selected packages as a software family.

The measured packages show strong practical infrastructure: all 28 provide installation instructions, installation automation, tutorials, user manuals, and API documentation, and only two installations broke during measurement. Weaknesses concentrate in formal artifacts: only one package documents expected user characteristics, and half do not identify a coding standard. Measured quality and community popularity diverge for several packages, so popularity indicators such as GitHub stars are not reliable proxies for engineering quality. Compared with prior studies of the state of the practice for medical imaging and Lattice Boltzmann method software, MPS software is at least as well equipped in repository artifacts and development tooling. A developer-interview component has been designed to investigate pain points and practices; its execution is left for future work.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Spencer Smith, for giving me this opportunity and for his patient guidance and feedback throughout this work. I am also grateful to Dr. Matthew Giamou, the domain expert for this study, for his help in selecting the software packages and for his advice on the scope, inclusion, and exclusion decisions.

I am thankful to Canada for the chance to pursue this degree, and to McMaster University for being my academic home over these two years. I would also like to thank everyone who helped and supported me along the way.

These two years have passed quickly. I have experienced a great deal and learned even more, and above all I am deeply grateful to my supervisor.

Contents

List of Figures

List of Tables

Notation, Definitions, and Abbreviations

Notation

s_i	Impression score (1–10) of package i on a given quality (Analytic Hierarchy Process)
a_{ij}	Pairwise comparison value between packages i and j on a given quality (Analytic Hierarchy Process)
w_i	Priority of package i on a given quality (Analytic Hierarchy Process)
c_k	Weight of quality k (Analytic Hierarchy Process)
p_i	Overall priority of package i across the nine qualities (Analytic Hierarchy Process)
n	Number of packages, i.e. the dimension of a pairwise-comparison matrix (Analytic Hierarchy Process)

Definitions

Alive package

In this report, a package is considered alive if it has received any commit or release in the 18 months preceding the measurement date.

Software family

A set of programs whose common properties are extensive enough that studying them collectively is advantageous before analysing individual members (Parnas, 1976).

State-of-the-Practice study

An assessment of the artifacts, tools, processes, and quality of existing software in a chosen domain, in contrast to a study that designs or implements new software.

Surface measure

An assessment based on shallow but systematic checks performed within a limited time budget rather than on exhaustive experimental evaluation.

Abbreviations

AHP	Analytic Hierarchy Process
AIST	National Institute of Advanced Industrial Science and Technology (Japan)

API	Application Programming Interface
CI	Continuous Integration
CI/CD	Continuous Integration and Continuous Delivery
CVC	Computer Vision Center (Barcelona)
DART	Dynamic Animation and Robotics Toolkit
DDS	Data Distribution Service
GIS	Geographic Information Systems
IIT	Istituto Italiano di Tecnologia (Italian Institute of Technology)
LBM	Lattice Boltzmann Method
LLM	Large Language Model
MI	Medical Imaging
MREB	McMaster Research Ethics Board
MJCF	MuJoCo XML configuration format
MPS	Motion Planning and Simulation
OMG	Object Management Group
OMPL	Open Motion Planning Library
OSS	Open Source Software
RL	Reinforcement Learning

ROS	Robot Operating System
SCC	Sloc, Cloc and Code (the source-code line-counting tool used for the repository measurements)
SDF	Simulation Description Format
SDK	Software Development Kit
SLAM	Simultaneous Localization and Mapping
SOFA	Simulation Open Framework Architecture
SOP	State-of-the-Practice
TRI	Toyota Research Institute
URDF	Unified Robot Description Format
YARP	Yet Another Robot Platform

Declaration of Academic Achievement

I, Ziyang Fang, declare that the research presented in this report was designed and carried out by me. This includes the candidate software discovery, the mention-count ranking, the repository-based measurements of the 28 selected packages, the commonality analysis, the comparison with other research software domains, and the writing of all chapters. Dr. Spencer Smith provided supervisory guidance on the methodology and on the structure and revision of the report. Dr. Matthew Giamou, as domain expert, reviewed the candidate software list and advised on scope, inclusion, and exclusion decisions. In the course of this work I made use of artificial-intelligence tools under my own direction. The exploratory trial reported in Appendix F is one such use; all validated measurements reported in Chapter 5 were performed and checked by me, and I take full responsibility for the content of this report.

Chapter 1

Introduction

Robotics software for motion planning and simulation (MPS) enables researchers and engineers to model robot kinematics and dynamics, simulate sensor data, plan collision-free trajectories, and test control strategies before deploying to physical hardware. An application of MPS software is shown in Figure 1.1.

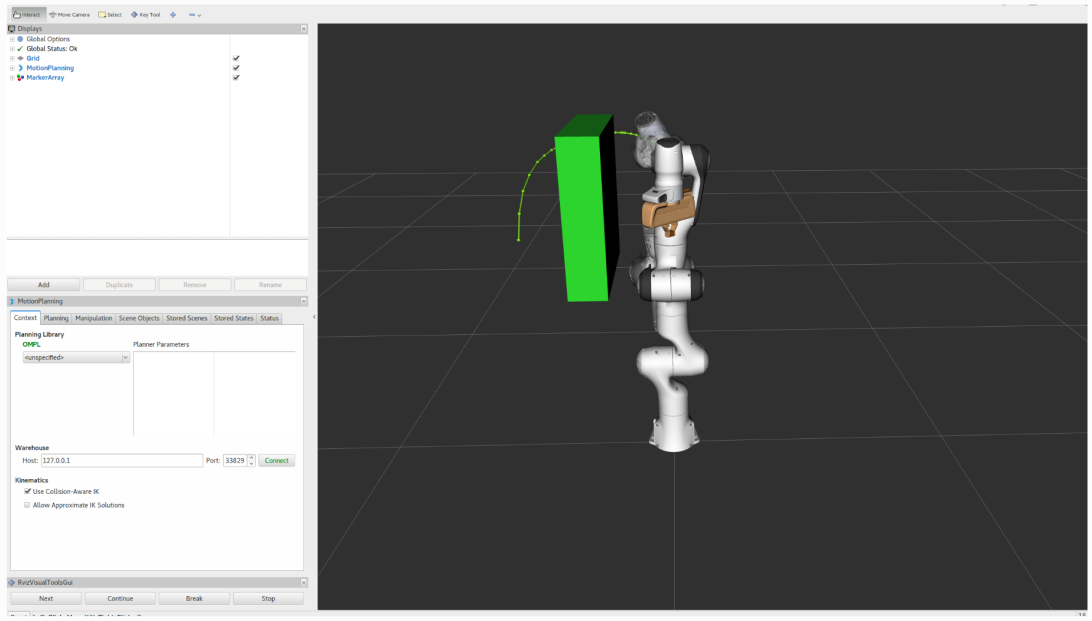

Figure 1.1: An illustrative path planning task: a manipulator computing a collision-free path around an obstacle.

In practice, this software is central to experimental reproducibility and to the transition from simulation to physical deployment, so qualities such as correctness, reliability, and long-term sustainability have direct consequences for both research and operational outcomes. A simulator that produces incorrect trajectories, or a planner that crashes under realistic inputs, can compromise experimental results.

Software engineers have developed methods and tools for improving software quality: version control, automated testing, and continuous integration are standard practice in professional contexts. However, a gap exists between these practices and what researchers actually do when developing scientific software. Scientists and engineers working on research software tend to learn software engineering skills informally, picking them up from colleagues or through trial and error rather than through formal training (Hannay et al., 2009). Hannay et al. (2009) observe that many scientists

show indifference to standard software engineering concepts, and Prabhu et al. (2011) report that more than half of the scientists they surveyed did not use a debugger. Because much of the software studied in this report originates in research settings, these concerns apply directly to the MPS domain.

Previous studies of the state of the practice have investigated this gap across several scientific software domains, most recently medical imaging (Smith et al., 2024a) and Lattice Boltzmann method software (Smith et al., 2024b); Section 2.3 reviews these studies. This report adapts the same methodology (Smith et al., 2021) to MPS software. The aim is to understand how MPS software holds up against established software engineering practices and how it compares with other research software domains. The report ranks the selected packages according to the defined quality attributes; these rankings describe the current state of the practice in software development, rather than serving as recommendations for any particular software package.

The remainder of this chapter states the research questions (Section 1.1), defines the scope (Section 1.2), overviews the methodology (Section 1.3), lists the contributions (Section 1.4), and outlines the remaining chapters (Section 1.5).

1.1 Research Questions

The following research questions guide this report.

RQ1. What software packages are candidates for the study of the state of the practice for MPS software?

RQ2. What is the ranking of the candidate packages from RQ1 with respect to the

software qualities of installability, correctness and verifiability, surface reliability, surface robustness, surface usability, maintainability, reusability, surface understandability, and visibility and transparency?

RQ3. How do MPS projects compare to other research software domains with respect to the artifacts present in their repositories?

RQ4. How do MPS projects compare to other research software domains with respect to the use of tools?

RQ5. How do MPS projects compare to other research software domains with respect to the principles, processes, and methodologies used?

RQ6. What are the pain points for developers working on MPS software projects?

RQ7. How do the pain points for MPS developers compare to the pain points for developers in other research software domains?

RQ8. For MPS developers, what specific best practices address the pain points identified in RQ6 and the software qualities mentioned in RQ2?

RQ9. What research software development practices, beyond those already used by MPS developers, address the pain points identified in RQ6?

RQ1–RQ5 are answered from public artifacts, while RQ6–RQ9 depend on the developer-interview component.

1.2 Scope

This report covers MPS software: robotics packages for motion planning, dynamics, physics, simulation, and supporting middleware. To be included, a package must expose sufficient public evidence, such as a source repository, documentation, issue tracker, releases, or project metadata, to support consistent repository-based measurement. Tools whose core components are proprietary or otherwise closed therefore fall outside the measured set even if they are widely used in robotics practice. Chapter 3 states the full inclusion and exclusion criteria and gives the rationale for individual tools that were considered and removed.

The selected packages play different roles within shared MPS workflows; Chapter 4 explains those roles before the measurement results are interpreted.

1.3 Overview of the Methodology

This report follows the methodology of Smith et al. (2021) (hereafter the Domain X methodology): it defines the MPS domain, identifies candidate packages from academic, community-curated, and auxiliary LLM-assisted sources, filters them to a measurable set, measures and ranks the selected packages from their public artifacts, analyzes them as a software family, compares the findings with prior domain studies, and prepares the developer-interview component. Chapter 3 details each step. The scope and exclusions were reviewed with a domain expert to reduce the risk of misrepresenting the MPS domain.

1.4 Contributions

This report makes the following contributions:

1. A scoped study of the state of the practice for MPS software, applying the methodology of Smith et al. (2021) to a new domain.

2. A transparent candidate discovery and selection process that combines academic sources, community-curated lists, and auxiliary LLM-assisted discovery, with explicit discussion of the role and limitations of each source type.

3. A public-artifact measurement dataset for 28 selected software packages, covering repository evidence, documentation, releases, issue trackers, testing and continuous-integration evidence, and project metadata, organized across the nine quality categories defined in the measurement template of Smith et al. (2021).

4. A commonality analysis identifying shared capabilities, variability points, and parameters of variation across the selected packages.

5. A comparison of MPS software with findings from prior studies of the state of the practice in other research software domains (Smith et al., 2024b,a, 2018c).

These contributions are descriptive and methodological.

1.5 Organization

The remainder of this report is organized as follows. The research questions are not confined to a single chapter; they are answered throughout the body, and each chapter

notes the research questions it addresses to maintain traceability back to Section 1.1.

- **Chapter 2** provides background on the MPS domain, prior studies of the state of the practice, the nine quality attributes, commonality analysis, and the Analytic Hierarchy Process.
- **Chapter 3** describes the methodology, from domain selection and candidate discovery to measurement, commonality analysis, and the interview component.
- **Chapter 4** presents the 28 selected packages as a software family and reports the commonality analysis.
- **Chapter 5** reports the measurement results, the quality-based rankings, and their comparison with community-facing indicators.
- **Chapter 6** compares MPS software with prior studies in other research software domains.
- **Chapter 7** sets out the developer-interview component for RQ6–RQ9.
- **Chapter 8** gives recommendations for future MPS software practice.
- **Chapter 9** discusses threats to validity.
- **Chapter 10** summarizes the study, answers the research questions, and identifies directions for future work.

Chapter 2

Background

This chapter sets out the concepts the rest of the report relies on: the MPS domain, the software licensing categories, state-of-the-practice methodology, the nine quality attributes, commonality analysis, the Analytic Hierarchy Process, and the role of repository-based measurement.

2.1 Robotics Software for Motion Planning and Simulation

MPS software supports the design, testing, evaluation, and deployment of robotic systems. Motion planning software generates feasible robot motions under constraints such as geometry, kinematics, dynamics, collisions, and task requirements (LaValle, 2006). Simulation software provides virtual environments for representing robots, sensors, actuators, controllers, and physical interactions before, or alongside, work with physical hardware (Collins et al., 2021).

The MPS domain in this report is defined to span more than a single class of software, so that the study includes enough packages to support a meaningful comparison. It covers full simulation environments, physics and dynamics engines, motion planning libraries, manipulation frameworks, and middleware. These classes of software are not interchangeable, but they are unified because they appear together in the same workflow: a simulator may call a physics engine for contact and dynamics, a planning stack may sit on top of a collision-checking library and communicate through middleware, and middleware may call planning, control, perception, and simulation. Figure 2.1 sketches these relationships among the four software roles; the report is explicit throughout about which role each package plays.

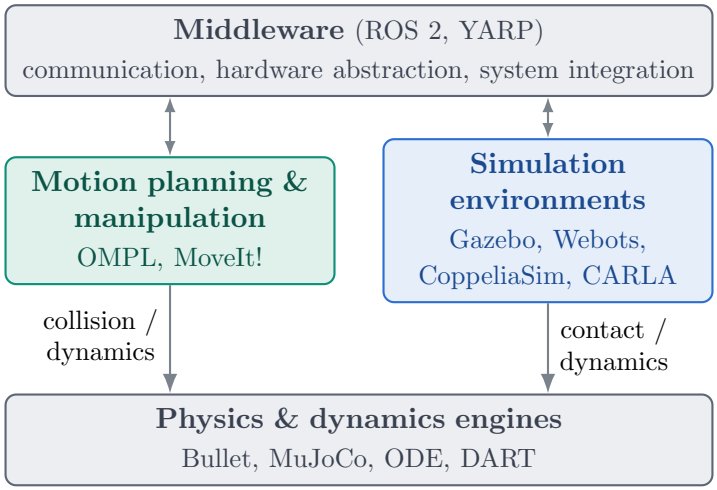

Figure 2.1: Representative MPS software roles. Box contents are example packages from the selected set.

2.2 Software Categories

Assessing software packages requires understanding the licensing model under which each package is distributed. In particular, the availability of source code determines

whether repository-based measurement is possible. Following the terminology used in prior studies of the state of the practice (Smith et al., 2024a,b), this report distinguishes three common software categories:

Open Source Software (OSS): For OSS, the source code is openly accessible. Users have the right to study, change, and distribute it under a license granted by the copyright holder. For many OSS projects, the development process relies on collaboration among contributors worldwide. Accessible source code exposes the underlying logic of software functions, the development practices used to produce them, and the flaws and potential risks in the final product.

Freeware: Freeware is software that can be used free of charge. Unlike OSS, the authors of freeware do not permit access to or modification of the source code. To many end users, the distinction between freeware and OSS may not matter.

Commercial Software: Commercial software is developed and sold by companies as a product. Typically, commercial software requires payment to access all features and does not include access to the source code. Some commercial software is available free of charge, but its source code is still not accessible.

This report assesses only software that fits under the Open Source Software category. This restriction is a methodological necessity, not a judgement about the quality or importance of freeware or commercial tools.

2.3 State-of-the-Practice Studies

A study of the state of the practice examines existing tools, artifacts, processes, and evidence in a chosen domain. The goal is not to build new software but to learn from

what already exists, by collecting and organizing public evidence in a systematic way. Claims about development activity, documentation, issue management, installation behaviour, or maintainability must be backed by observable project artifacts. When evidence is missing or ambiguous, the gap is recorded rather than filled with assumptions.

The methodology used in this report is based on "Methodology for Assessing the State of the Practice for Domain X" by Smith et al. (2021). It is generic enough to apply to any research software domain and is designed to be completed by a small team within a bounded time budget (Section 3.7.3). The prescribed process runs from domain selection through candidate filtering, template-based measurement, developer interviews, and AHP-based ranking to answers to a set of research questions; Chapter 3 enumerates the steps as applied to MPS.

The methodology builds on prior assessments of the state of the practice conducted across several research software domains: Geographic Information Systems (Smith et al., 2018a), Mesh Generators (Smith et al., 2016), Seismology software (Smith et al., 2018c), and Statistical software for psychology (Smith et al., 2018b). More recently, the methodology was refined and applied to Medical Imaging (MI) software (Smith et al., 2024a) and Lattice Boltzmann Method (LBM) software (Smith et al., 2024b; Michalski, 2021). These studies are the primary structural and stylistic references for this report.

Practitioner-facing research software checklists provide complementary guidance on themes such as source control, licensing, documentation, testing, automation,

distribution, and governance (Acton, 2026). These themes overlap with several Domain X quality categories, but this report uses the Domain X template as its measurement instrument.

2.4 Software Quality Attributes

This report treats software quality operationally: a package exhibits quality to the extent that observable attributes support dependable installation, use, modification, reuse, and understanding. Following common practice in software engineering, quality is decomposed into distinct attributes, often called "ilities" (Smith et al., 2021). This report assesses nine quality attributes drawn from the measurement template in Appendix B of Smith et al. (2021); this template is reproduced, as adapted for this study, in Appendix A of this report. Several of the qualities use the word "surface" to indicate that the assessment is based on shallow but systematic checks performed within a limited time budget, not on exhaustive experimental evaluation (Smith et al., 2024a).

Installability: The effort required for the installation and uninstallation of software in a specified environment. Measured through the presence and quality of installation instructions, the number of steps required, the availability of automation, the handling of dependencies, and the presence of validation procedures.

Correctness and Verifiability: A program is correct if it matches its specification, which may be stated explicitly or implicitly. The related quality of verifiability is the ease with which the software components or the integrated product can be checked to demonstrate correctness. Measured through the presence of

requirements or theory documentation, techniques used to build confidence in correctness (such as automated testing, assertions, or continuous integration), and the availability and quality of tutorials with expected outputs.

Surface Reliability: The probability of failure-free operation of a computer program in a specified environment for a specified time. Surface reliability is assessed through whether the software breaks during installation or initial tutorial use, whether descriptive error messages are displayed, and whether recovery is possible.

Surface Robustness: Software possesses robustness if it behaves reasonably when it encounters circumstances not anticipated in the requirements specification, and when users violate the assumptions in its requirements specification. Surface robustness is assessed through whether the software handles unexpected or malformed input gracefully, including edge cases such as modified line endings in text input files.

Surface Usability: The extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use. Surface usability is assessed through the presence of getting-started tutorials, user manuals, documented user characteristics, and the availability and variety of user support channels.

Maintainability: The effort with which a software system or component can be modified to correct faults, improve performance or other attributes, or satisfy new requirements. Assessed through version information, contribution and

review guidance, available non-code artifacts, issue tracking practices, the percentage of identified issues that are closed, the percentage of code that consists of comments, and the version control system in use.

Reusability: The extent to which a software component can be used with or without adaptation in a problem solution other than the one for which it was originally developed. Assessed through the number of code files and the presence and quality of API documentation.

Surface Understandability: The capability of the software product to enable the user to understand whether the software is suitable, and how it can be used for particular tasks and conditions of use. Surface understandability is assessed through a review of randomly selected source files, examining indentation consistency, coding standards, identifier quality, use of named constants, comment clarity, algorithm references, parameter ordering, and modularization.

Visibility and Transparency: The extent to which all steps of a software development process and the current status of that process are conveyed clearly to external observers. Assessed through whether the development process is defined and documented, whether the development environment is documented, and whether release notes are published.

These nine qualities organize public evidence into comparable categories. They are not complete evaluations of each package. Scores may vary with the use case, platform, user background, or measurement date. Later chapters therefore report both the measurements and their limitations.

2.5 Commonality Analysis

A domain analysis consists of systematically identifying and documenting the commonalities, variabilities, and parameters of variation of a software family (Weiss, 1998). A software family is defined as "a set of programs whose common properties are so extensive that it is advantageous to study the common properties of the programs before analyzing individual members" (Parnas, 1976). Weiss (1998) defines commonality analysis as an approach to defining a family by identifying three kinds of information:

Commonalities: Goals, theories, models, definitions, and assumptions that are shared across family members. For example, all robotics simulators share the goal of representing robot kinematics and dynamics in a virtual environment.

Variabilities: Goals, theories, models, definitions, and assumptions that differ between family members. For example, simulators may differ in the physics engines they use, the sensor models they support, or the robot platforms they can represent.

Parameters of Variation: The possible values for each variability, along with their binding time. Binding time records when a particular variation is fixed: at specification time (when the software is designed), at build time (when the software is compiled), at configuration time (when the software is set up for use), or at run time (when the software is executing).

Commonality analysis was added to the Domain X methodology after early applications revealed that software packages within a domain can vary considerably, even

when they appear to belong to the same family (Smith et al., 2024b). Without it, comparisons risk treating fundamentally different types of software as if they were interchangeable. The analysis records where the packages do similar things and where their roles diverge, so a low score on, for example, installability means different things for a full simulator and a focused dynamics library.

Chapter 4 presents this commonality analysis for the selected packages.

2.6 Analytic Hierarchy Process

The Domain X methodology ranks the 28 software packages with the Analytic Hierarchy Process (AHP), a pairwise-comparison technique introduced by Saaty (1980). Each package has one impression score, from 1 to 10, for each of the nine qualities. AHP combines these scores into a single overall priority per package, by which the packages are ordered, following the approach of Smith et al. (2016).

AHP is applied to one quality at a time. For a given quality, let the package scores be $s_1, \ldots, s_n$. Each ordered pair of packages (i, j) is given a comparison value on Saaty's 1–9 scale, read directly from the two scores:

$$a_{ij} = \begin{cases} \min\{9,\ s_i - s_j + 1\}, & s_i \geq s_j, \\ 1 \big/ \min\{9,\ s_j - s_i + 1\}, & s_i < s_j. \end{cases} \tag{2.6.1}$$

Here $a_{ij} = 1$ means the two packages are judged equal on the quality, and $a_{ij} = 9$ is the maximum of the scale, used when package i's score far exceeds package j's. The values form an $n \times n$ reciprocal matrix A (with $n = 28$ packages), with $a_{ji} = 1/a_{ij}$. The priority of each package on the quality follows the standard AHP averaging

method (Saaty, 1980): each column of A is normalized to sum to one, and the entries of each row are averaged,

$$w_i = \frac{1}{n} \sum_{j=1}^{n} \frac{a_{ij}}{\sum_{l=1}^{n} a_{lj}}. \tag{2.6.2}$$

The priorities $w_1, \ldots, w_n$ are non-negative and sum to one, and w_i is the priority of package i on that quality; sorting the packages by w_i gives their ranking on that quality. Repeating the procedure for all nine qualities gives nine priority vectors $\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(9)}$, and hence nine per-quality rankings.

Combining the nine per-quality rankings into a single overall ranking requires weights $c_1, \ldots, c_9$ that express the importance of each quality; these weights are obtained by applying the same AHP pairwise-comparison procedure to the qualities themselves. A package's overall priority is the weighted sum of its per-quality priorities:

$$p_i = \sum_{k=1}^{9} c_k \, w_i^{(k)}, \qquad \sum_{k=1}^{9} c_k = 1, \tag{2.6.3}$$

The packages are ranked by p_i.

AHP is an analysis aid for cross-quality comparison under a stated weighting, not a universal recommendation of any particular package. Selecting a software package for a specific application would involve finding the weights appropriate for that application. The weighting adopted in this report, and the resulting ranking, are stated in Section 3.7.4 and Chapter 5.

2.7 Open Source and Repository-Based Measurement

The measurement process in this report depends on evidence from public artifacts. Public repositories can reveal commit history, release activity, issue tracking, documentation, source code organization, testing practices, dependencies, and community interaction. Public documentation and project websites can add evidence about installation procedures, usage patterns, supported platforms, and design goals.

Repository-based measurement has inherent limitations. Repository metadata represents a snapshot in time: last commit dates, issue counts, release information, and pull request activity can change after measurement. Some projects distribute their development activity across multiple repositories or use external issue trackers, making it difficult to capture a complete picture from any single repository. The interpretation of repository metrics requires care: a high number of open issues may indicate an active user community rather than poor maintenance, and a low commit frequency may reflect project maturity rather than abandonment.

The measurement record therefore documents measurement dates, repository choices, and interpretation rules. Proprietary tools and tools with closed core components may be important in robotics practice but fall outside this procedure (Section 2.2); Chapter 3 discusses the specific inclusion and exclusion decisions that follow.

Chapter 3

Methodology

This chapter presents the study methodology and addresses RQ1 (software candidate identification), documenting how the candidate set was assembled. The methodology follows Smith et al. (2021), adapted to the MPS domain, in the following steps:

1. **Select the research domain** (Section 3.1).
2. **Engage a domain expert** who participates in the assessment and later reviews the candidate list (Section 3.2).
3. **Identify candidate software** from academic survey papers, community-curated lists, and LLM-assisted lists (Section 3.3).
4. **Rank the candidates** by mention count (Section 3.4).
5. **Filter the ranked list** to the selected set of 28 packages (Sections 3.5 and 3.6).
6. **Measure each package** with the Domain X template and rank the results with AHP (Section 3.7).

7. **Analyze commonalities** across the selected packages (Section 3.8).

8. **Interview developers** about their development practices and pain points (Section 3.9).

3.1 Domain Selection

The first step of the Domain X methodology is to identify a suitable research software domain. To be applicable for the methodology, the chosen domain must meet four criteria (Smith et al., 2021): (i) it must have a well-defined and stable theoretical underpinning, ideally supported by standard textbooks; (ii) there must be a community of researchers and practitioners studying the domain; (iii) the software packages in the domain must include open-source options; and (iv) a preliminary search should suggest that there are close to 30 candidate software packages that qualify as research software.

The selected domain is *MPS (motion planning and simulation)*. Motion planning and simulation are established subfields of robotics with well-defined theoretical foundations (LaValle, 2006; Choset et al., 2005). There is an active international research community, regular conferences (such as ICRA, IROS, and RSS), and a substantial body of published work. A preliminary search confirmed the existence of numerous open-source software packages across the MPS software roles defined in Section 2.1.

The domain boundary follows the MPS scope defined in Chapter 2, Section 2.1, rather than a simulator-only scope.

Earlier project discussions considered a narrower domain focused on inverse kinematics software. After consultation with the supervisor and domain expert, the scope

was broadened to the MPS domain, which provides a richer set of candidate packages and better reflects the interconnected nature of robotics software ecosystems.

3.2 Interaction with the Domain Expert

The Domain X methodology emphasizes that a domain expert is an essential member of the assessment team. Pitfalls exist if non-experts attempt to produce an authoritative list of software or definitively rank packages without domain knowledge. Non-experts must rely on information available online, which has two known drawbacks: (i) online resources may contain false or inaccurate information; and (ii) online resources may omit relevant information that is so ingrained with experts that nobody thinks to record it explicitly (Smith et al., 2021).

The expert need not be a software developer, only a knowledgeable user of domain software, and the methodology is designed to minimize the demand on their time (Smith et al., 2024a).

For this report, the domain expert is Dr. Matthew Giamou of the Department of Computing and Software at McMaster University, whose research includes robotics, motion planning, and state estimation. His review of the candidate list and the filtering outcome is reported in Section 3.6, after the preliminary list has been assembled.

3.3 Candidate Software Discovery

Candidate software packages were identified from three types of sources: academic survey papers (Section 3.3.1), community-curated software lists (Section 3.3.2), and LLM-generated software lists (Section 3.3.3). Using multiple source types reduces

dependence on any single search method and provides broader coverage of the MPS domain. The collected names were consolidated to merge equivalent entries (for example, "V-REP" and "CoppeliaSim" refer to the same package), and the consolidated list was used as input to the mention-count ranking described in Section 3.4.

3.3.1 Academic Survey Papers

Academic sources identify software packages that appear in research discussions of robotics simulators, simulation tools, and related challenges. These sources included survey papers and comparative studies, such as Harris and Conrad (2011), Collins et al. (2021), Körber et al. (2021), and Farley et al. (2022), that discuss packages such as Gazebo, Webots, V-REP/CoppeliaSim, MuJoCo, and PyBullet in the context of robotics research. The full list of academic sources used for mention counting appears in Appendix B.

Academic sources have known limitations as the sole basis for candidate discovery. Survey papers tend to emphasize packages relevant to a specific research question, robot type, or time period; some active tools may not appear in any of the surveys consulted, while older tools sometimes remain visible because they were prominent when a survey was written.

3.3.2 Community-Curated Sources

Community-curated sources broaden the view of robotics software practice beyond what appears in academic surveys. The source catalogue included GitHub Topic pages, "awesome" lists (curated collections such as `awesome-robotics-libraries`, `awesome-robotic-tooling`, and `awesome-motion-planning`), best-of lists such as

`best-of-robot-simulators`, and other community-maintained aggregations. Sources covered topics including motion planning, robotics simulation, robotics libraries, ROS 2 ecosystem tools, multibody dynamics simulation, and general robotic tooling. The GitHub Topic pages were manually inspected as dynamic community aggregations, while the static curated-list pages were preserved as copied raw web material where available. Appendix B lists the community-curated sources individually.

Community-curated sources vary in maintenance quality, inclusion criteria, and update frequency, so they were used as discovery aids rather than as indicators of software quality.

3.3.3 LLM-Assisted Sources

LLM-generated software lists were used as an auxiliary source for candidate discovery. This modifies the original Domain X methodology, which prescribes search engine queries and domain expert consultation as the primary discovery methods. The modification is recorded explicitly so that the methodology remains transparent and reproducible in principle. The model, prompt, and query date for each LLM-generated list are recorded in Appendix B.

The reason for including LLM-assisted discovery is practical: a large language model acts as a broad aggregator over existing public information, analogous in effect to a wide search over indexed web content. LLM output can also be incomplete, outdated, or incorrect, so names obtained from LLM-generated lists were consolidated, filtered, and verified against public evidence before any claims in this report were based on them.

3.4 Mention Count Ranking

After consolidating equivalent names across the collected sources, each unique software package was counted once per source to produce a mention count. A package that appeared in five academic papers and three community-curated lists would receive a mention count of eight. The consolidated ranking was recorded during the discovery phase and is a transparent input to the filtering process.

Consolidation produced 295 unique candidate names. The top entries in the consolidated ranking include Gazebo (22 mentions), Webots (16), Bullet/PyBullet (16), CoppeliaSim/V-REP (13), MuJoCo (12), ODE (10), ROS 2 (10), and OpenRAVE (9). Appendix C records every candidate with at least four mentions, together with its mention count and filtering outcome, and Appendix B lists the counted sources. The full ranked list of all 295 names, including packages that did not survive the filtering step described in Section 3.5, is preserved in the supplementary project materials that accompany this report (a consolidated ranking spreadsheet and a companion source catalogue).[1]

Mention count is a source-appearance signal and must be interpreted accordingly. It does not measure software quality, user adoption, research impact, maintainability, or the overall suitability of any particular software package. A package may appear often because it is historically established or broadly useful, while a newer or more specialized package may be under-represented even when it is technically strong.

[1] The supplementary project materials are available in the project repository at `https://github.com/smiths/AIMSS/tree/master/StateOfPractice/DomainMeasurements/MPS-Ryan`.

3.5 Filtering Criteria

The ranked candidate list was filtered to identify packages suitable for repository-based measurement of the state of the practice. In the Domain X methodology, the initial filters are scope, usage, and age, applied in priority order until the list reaches a manageable size of approximately 30 packages (Smith et al., 2021). In this report, these filters are implemented through three criteria: open-source availability, repository-based measurability, and activity and maintenance status.

3.5.1 Open Source Availability

Open-source availability was required because the study depends on public software artifacts for measurement. The Domain X candidate criteria require that source code be viewable and express a preference for GitHub-style repositories from which empirical measures can be collected (Smith et al., 2021). Source code, repository history, issue trackers, documentation, and release information provide the evidence base for the measurements reported in Chapter 5.

This criterion led to the exclusion of several prominent robotics tools whose core components are not open source:

- **MATLAB and Simulink** are major platforms in both robotics research and industry, widely used for algorithm development, control design, and simulation.
- **Unity** is a widely used game engine with robotics simulation capabilities.
- **NVIDIA Isaac Sim** builds on NVIDIA Omniverse and provides advanced robotics simulation features.

- **RaiSim** is a high-performance physics simulator for robotics.

The exclusions above are methodological constraints, not judgements about the quality or importance of the excluded tools; the resulting coverage limit is discussed in Chapter 9.

3.5.2 Repository-Based Measurability

Repository-based measurability requires that a package have sufficient public evidence to support the measurement process described in Section 3.7. Relevant evidence includes source repositories, commit history, issue data, documentation, release information, build or installation instructions, test artifacts, and project metadata. Packages without sufficient public evidence cannot be measured consistently and were excluded.

This criterion also governs how multi-repository projects are handled. Some robotics software systems are distributed across multiple repositories or GitHub organizations. In such cases, the repositories most representative of the core software were selected for measurement, and this choice was documented as part of the measurement record. The same principle applies to projects that have changed names, migrated repositories, or spawned successor projects.

3.5.3 Activity and Maintenance Status

Activity and maintenance status were considered because the study focuses on contemporary software practice. In the Domain X methodology, age is one of the initial filtering criteria, measured by the last date when a change was made, with exceptions for older packages that remain highly recommended and currently in use (Smith et al.,

2021). Tools that are no longer maintained provide weaker evidence about current development practices.

The primary activity indicator used in this report is the Domain X alive/dead rule: a package is considered alive if it has received commits or version releases within the last 18 months from the measurement date. This indicator was used as a weighted signal rather than as an automatic cutoff: a package was set aside on activity grounds only when prolonged inactivity coincided with the tool no longer being prominent or having been superseded by an actively maintained alternative. On that basis, the following packages were excluded:

- **MRDS** (Microsoft Robotics Developer Studio) has not received updates since 2012 and is no longer supported by Microsoft.
- **USARSim** (Unified System for Automation and Robot Simulation) is no longer actively maintained and does not represent current practice.

Four packages (OpenRAVE, MORSE, GraspIt!, and Flightmare) had become inactive by the measurement date but remain prominent and widely referenced, and so were retained in the measured set; their alive/dead status is recorded and reported as a measured outcome in Chapter 5 (Section 5.1), so that the study can examine how software quality persists after development activity declines.

3.5.4 Scope and Coverage Filters

Two algorithm-level reference implementations were excluded to keep the selected packages methodologically coherent rather than for quality reasons. **GPMP2** (Gaussian Process Motion Planner 2) and **CHOMP** (Covariant Hamiltonian Optimization

for Motion Planning) are optimization-based motion planning algorithms with public reference implementations. They are relevant to motion planning research but are narrower in scope than the general-purpose simulators, physics engines, planning frameworks, and middleware tools that dominate the candidate list. Including them alongside Gazebo, MoveIt!, or ROS 2 would force a comparison between full software stacks and single-algorithm libraries, which would distort the commonality analysis and the cross-package measurements.

Filtering decisions use a measurement-time snapshot of repository activity and release status; these signals can change afterward, so the measurement date is recorded with the results (Chapter 5).

3.6 Final Software Set

After ranking by mention count and applying the filtering criteria described above, the selected set consists of 28 software packages. Appendix C records the disposition of the leading candidates, and Appendix D details the excluded packages. Figure 3.1 summarizes the full selection pipeline, from the three discovery sources through the mention-count ranking and the filtering criteria to the final set. Table 3.1 lists the packages alphabetically with their primary roles in the robotics software ecosystem.

The domain expert, Dr. Matthew Giamou, reviewed the candidate list and the filtering outcome. He confirmed that the list was reasonable and that most of the shortlisted packages were familiar to him, and he noted that ROS 2, MoveIt!, and OMPL are not full simulators but fit within a coherent narrative when their roles are explained carefully. He agreed that excluding MATLAB/Simulink, Unity, NVIDIA

Isaac Sim, and RaiSim on open-source grounds (Section 3.5.1) would not misrepresent the MPS domain; MRDS and USARSim were filtered on activity grounds (Section 3.5.3) rather than by a separate expert decision.

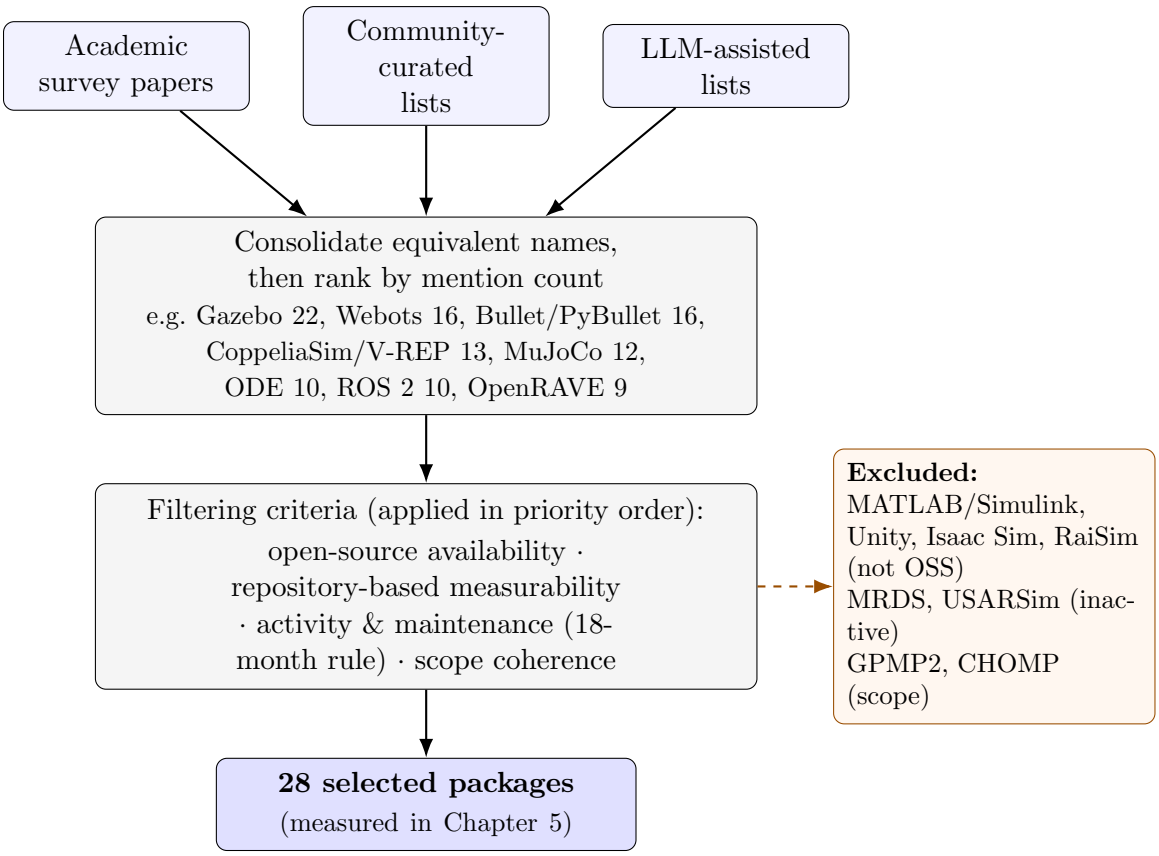

Figure 3.1: Candidate selection pipeline, from the three discovery sources to the 28 measured packages. The activity criterion is a weighted signal, not a hard cutoff (Section 3.5.3).

Table 3.1: Final software set: primary role, repository release date, last commit date, and a representative publication. Repository dates are a May 2026 snapshot; the "Released" date is the first release in the measured public repository, which for renamed, re-licensed, or migrated projects (e.g. MuJoCo, PhysX, Webots) reflects that later event rather than the project's original creation (Section 5.1). "—" marks packages for which no representative publication was selected; the listed publications are the most representative reference available and are not necessarily peer reviewed.

Software	**Primary role**	**Released**	**Updated**	**Publication**
AirSim	Simulator (autonomous vehicles)	2017-02-13	2026-03-15	Shah et al. (2018)
Bullet/PyBullet	Physics engine	2006-05-25	2025-10-21	Coumans and Bai (2016–2021)
CARLA	Simulator (autonomous driving)	2017-03-03	2026-05-11	Dosovitskiy et al. (2017)
CoppeliaSim (V-REP)	Simulator (general robotics)	2019-09-27	2026-05-11	Rohmer et al. (2013)
DART	Dynamics library	2011-04-01	2026-05-10	Lee et al. (2018)
Drake	Toolbox (dynamics, planning, control)	2010-08-03	2026-05-11	Tedrake and the Drake Development Team (2019)
Flightmare	Simulator (quadrotor)	2020-07-14	2023-05-15	Song et al. (2021)
Gazebo	Simulator (general robotics)	2014-04-07	2026-05-09	Koenig and Howard (2004)
GraspIt!	Manipulation (grasping)	2010-10-01	2020-07-10	Miller and Allen (2004)
Habitat	Simulator (embodied AI)	2019-02-25	2026-05-07	Savva et al. (2019)
MORSE	Simulator (academic robotics)	2009-10-30	2022-03-22	Echeverria et al. (2011)
MoveIt!	Manipulation (motion planning)	2011-10-11	2026-03-23	Coleman et al. (2014)
MRPT	Library (mobile robotics)	2010-06-03	2026-05-10	—
MuJoCo	Physics engine	2021-10-18	2026-05-11	Todorov et al. (2012)
Newton Dynamics	Physics engine	2013-05-06	2026-05-11	—
ODE (Open Dynamics Engine)	Physics engine	2001-03-04	2026-04-14	—
OMPL	Motion planning library	2010-04-26	2026-05-08	Şucan et al. (2012)
OpenRAVE	Planning framework	2009-08-15	2024-08-16	Diankov and Kuffner (2008)

Software	Primary role	Released	Updated	Publication
OpenRTM-aist	Middleware (RT components)	2005-05-12	2026-05-11	Ando et al. (2005)
OROCOS	Control framework	2007-02-13	2026-05-07	Bruyninckx (2001)
PhysX (open-source SDK)	Physics engine	2022-11-08	2026-04-13	—
Pinocchio	Dynamics library	2014-10-07	2026-05-11	Carpentier et al. (2019)
Project Chrono	Multi-physics simulation	2010-02-23	2026-05-11	Tasora et al. (2016)
ROS 2	Middleware and SDK	2015-02-24	2026-04-29	Macenski et al. (2022)
Simbody	Multibody dynamics library	2005-07-26	2026-03-19	Sherman et al. (2011)
SOFA	Simulation framework	2005-12-08	2026-05-08	Faure et al. (2012)
Webots	Simulator (general robotics)	2018-11-05	2026-05-04	Michel (2004)
YARP	Middleware (humanoid robotics)	2006-03-10	2026-04-25	Metta et al. (2006)

The role column is necessary because the packages are not direct substitutes; Chapter 4 interprets the boundary cases.

3.7 Measurement Procedure

The measurement procedure follows the Domain X method for collecting structured evidence about software packages in a selected domain (Smith et al., 2021). For each of the 28 selected packages, the study gathered the public evidence described in Section 3.5.2, and the measurement record identifies the specific repository or repositories measured.

3.7.1 Measurement Template

The primary measurement instrument is the measurement template from Appendix B of Smith et al. (2021); the version used in this study is reproduced in Appendix A. The template contains approximately 108 fields when supplementary entries (free-text "additional comments" fields after each section and the GitHub-style and GitStats/SCC repository metric panels) are counted alongside the substantive questions. The substantive questions are grouped into the following sections:

1. **Summary Information** (17 questions): software name, URL, affiliation, purpose, number of developers, funding, initial release date, last commit date, status, license, platforms, software category, development model, publications, source code URL, programming languages, and evidence of performance consideration.

2. **Installability** (14 questions): installation instructions (presence, single-document organization, linear ordering, completeness, and assumed pre-installed dependencies), compatible operating system versions, installation automation, error handling, installation validation procedure, number of installation steps, operating system used during measurement, dependencies (presence of guidance and required version listing), and uninstall behaviour (problems and residue).

3. **Correctness and Verifiability** (8 questions): requirements or theory documentation, correctness techniques, tutorial availability and quality, unit tests, and continuous integration evidence.

4. **Surface Reliability** (5 questions): installation and tutorial breakage, error messages, and recoverability.

5. **Surface Robustness** (2 questions): handling of unexpected input and edge cases.

6. **Surface Usability** (4 questions): tutorial, user manual, user characteristics, support model.

7. **Maintainability** (7 questions): version number, contribution guidance, available artifacts, issue tracking, issue closure rate, comment percentage, and version control.

8. **Reusability** (2 questions): number of code files and API documentation.

9. **Surface Understandability** (8 questions): code formatting, coding standards, identifier quality, constant usage, comment clarity, algorithm references, parameter ordering, and modularization.

10. **Visibility and Transparency** (4 questions): development process definition, process documentation, development environment documentation, and release notes.

Each quality section concludes with an overall impression score on a scale of 1 to 10, assigned using the scoring guidelines of the impression calculator in Appendix C of Smith et al. (2021). The impression scores summarize more than the recorded checklist answers alone: they also reflect observations made during measurement, such as the quality and currency of documentation, the clarity of error messages, and the overall installation and tutorial experience, which the mostly binary checklist fields do not capture. Two packages with identical checklist answers can therefore receive different impression scores; Chapter 9 discusses the subjectivity this introduces.

3.7.2 Repository Metrics

Repository metrics are collected separately from the quality impression scores. Three categories of repository data are gathered:

1. **GitHub-style metrics:** stars, forks, watchers, open pull requests, closed pull requests, number of developers, open issues, closed issues, initial release date, and last commit date.

2. **GitStats-style metrics:** number of text-based files, number of binary files, total lines, lines added, lines deleted, total commits, commits by year (last 5 years), and commits by month (last 12 months).

3. **SCC-style metrics:** number of text-based files, total lines, code lines, comment lines, and blank lines.

From these raw data, three processed measures are calculated:

1. **Alive/dead status:** computed under the 18-month alive/dead rule (Section 3.5.3).

2. **Percentage of identified issues that are closed:** computed as closed issues divided by total identified issues, using GitHub issue data where available.

3. **Percentage of code that is comments:** computed with the SCC tool's line-based method, as comment lines divided by total lines.

3.7.3 Measurement Execution

The Domain X methodology recommends performing installation-based measurements in a controlled environment, typically a virtual machine, to ensure a clean

and reproducible starting state (Smith et al., 2021). For this report, installation measurements were performed on a Linux-based virtual machine. The operating system, installation steps, dependencies, and any issues encountered were recorded for each package.

The methodology also recommends a time budget to keep the overall measurement workload feasible. The full assessment is estimated at roughly 173 person-hours per domain, against a stated goal of about one person-month (160 hours), with 1 to 4 hours allocated per package for template completion and up to 2 hours for installation (Smith et al., 2021). Measurement was conducted during May 2026. All time-sensitive data (last commit dates, issue counts, stars, forks, etc.) should be understood as snapshots taken during this period.

One further interpretation rule applies: **evidence that performance was considered** is assessed by searching software artifacts for explicit references to speed, storage, throughput, performance optimization, parallelism, multi-core processing, or similar considerations.

3.7.4 Ranking with AHP

After the quality measures are collected, the Domain X methodology ranks the packages with the Analytic Hierarchy Process (AHP), as defined in Section 2.6, Equations (2.6.1)–(2.6.3).

In this report, AHP is applied to the nine quality impression scores for the 28 packages. Because no criterion-weight elicitation was carried out, the qualities are weighted equally ($c_k = 1/9$), so the overall priority of Equation (2.6.3) reduces to the mean of the nine per-quality priorities. Chapter 5 reports the resulting equal-weight

AHP ranking alongside the per-package quality-mean ranking and the comparison with community indicators. The robustness of this ranking to the precise impression scores is assessed by perturbing each score by a random amount of up to ±10% and recomputing the ranking, following the sensitivity check of the LBM assessment (Smith et al., 2024b).

3.8 Commonality Analysis Procedure

The commonality analysis identifies shared and variable capabilities across the selected software packages, treating them as a software family (Section 2.5). It supplements the repository-based metrics by describing what the packages do and how their roles differ, providing context for interpreting the measurement results reported in Chapter 5.

Following the Domain X methodology, the analysis proceeds in six steps:

1. **Write an introduction** to the domain and the software family.
2. **Write an overview of domain knowledge,** summarizing the key concepts and terminology relevant to MPS software.
3. **List commonalities:** identify goals, assumptions, functions, artifacts, and domain concepts that recur across multiple family members.
4. **List variabilities:** identify dimensions on which family members differ, such as software role, supported platforms, licensing, programming language interfaces, documentation style, and intended use cases.

5. **List parameters of variation:** for each variability, describe the possible values.

6. **Add terminology, definitions, and acronyms** that are standard in the domain, to ensure that the analysis is accessible to readers outside robotics.

The procedure first groups the selected packages into broad categories based on their primary roles: robotics simulators, physics and dynamics engines, and planning or middleware packages. It then identifies capabilities that appear across multiple categories, such as support for robot model representation, simulated environments, motion planning workflows, dynamics computation, and integration with robotics ecosystems.

Because the selected packages span multiple roles, the commonality analysis does not force all packages into a single comparison template. The analysis is based on checked evidence from documentation, repositories, publications, and other reliable project materials. Capabilities that could not be verified against these materials are not reported.

3.9 Interview Component

The Domain X methodology includes a developer survey component designed to capture information that is not visible in public repositories: the development process as experienced by practitioners, the pain points they encounter, their perceptions of software quality threats, and the strategies they use to address those threats (Smith et al., 2021). The methodology prescribes semi-structured interviews based on a

list of 18 questions covering developer background, project organization, user understanding, development obstacles (past and present), documentation practices, and specific software qualities including maintainability, understandability, usability, and reproducibility.

The interview component follows a McMaster Research Ethics Board (MREB) protocol. The study received MREB ethics clearance under project #8215 on 2026-06-22; recruitment of developers began after clearance. Only evidence supported by approved records and consented summaries is reported. Interview participants are software developers, maintainers, or programmers associated with the studied packages. The interview guide is reproduced in Appendix E.

Chapter 7 addresses the developer interview component and research questions RQ6–RQ9.

The methodology recommends sending interview requests to developers from all packages in the study set to avoid selection bias. Prior applications of the methodology report response rates roughly in the 15%–30% range (Smith et al., 2021, 2024a); the exact rate for this report will depend on outreach timing and the response practices of the contacted developers. Interview contacts are typically identified through project websites, code repositories, publications, or institutional biography pages.

Chapter 4

Selected Software and Commonality Analysis

This chapter characterizes the final 28 packages (Section 3.6) for RQ1 and gives the commonality context used later when interpreting the RQ2 measurements. The commonality analysis follows the Weiss (1998) procedure described in Section 3.8; capabilities are reported only where they could be verified against the project materials.

4.1 Overview of Selected Software

The selected packages come from a mix of development settings—academic research groups, open-source foundations and companies, corporate research labs, and smaller communities or individual maintainers—detailed under V9 (Section 4.4).

The packages span a period of active development from 2001 (ODE's initial release) to the present, with most showing recent repository activity as of the May 2026

measurement date. Licensing, language, and platform details are reported with the measurement data in Section 5.1 (Table 5.1).

4.2 Functional Software Roles

The selected packages can be grouped into three broad functional roles by their primary function in a robotics workflow. These roles are not strict boundaries—some packages span several (see Section 4.6)—but they structure the commonality analysis.

4.2.1 Robotics Simulators

Robotics simulators provide virtual environments for representing robots, worlds, sensors, and actuators, letting users test algorithms, validate designs, and collect synthetic data before deploying to hardware. They typically offer graphical rendering, physics simulation, sensor models, and programmatic interfaces for controlling simulated robots. The selected packages primarily classified as simulators are:

- **Gazebo** (including the Ignition/Gazebo Sim lineage) is a long-established open-source 3D robotics simulator developed by Open Robotics. It provides physics simulation through multiple backends, rendering, sensor models, and ROS integration.
- **Webots** is a multi-platform robot simulator maintained by Cyberbotics. It supports modelling, programming, and simulating robots, vehicles, and mechanical systems, with a large library of pre-built robot models.
- **CoppeliaSim** (formerly V-REP) is a versatile robot simulation framework developed by Coppelia Robotics. It supports rapid algorithm development, factory

automation simulation, prototyping, verification, and digital twin applications.

- **AirSim** is an open-source simulator for autonomous vehicles (drones and ground vehicles) built on Unreal Engine and developed by Microsoft Research. The measured repository is `microsoft/AirSim`; under the May 2026 18-month activity rule it is classified as alive (last commit 2026-03-15), but the README has announced since 2022 that development has ended ("no further updates will be made, effective immediately") and points users to Project AirSim. Post-2022 commits are README-level only, so the alive label is a repository snapshot, not a claim of continuing stewardship.

- **CARLA** is an open-source autonomous driving simulator built on Unreal Engine and developed by the Computer Vision Center in Barcelona and Intel. It provides photorealistic urban environments and a broad sensor suite for autonomous driving research.

- **MORSE** (Modular OpenRobots Simulation Engine) is a generic academic robotic simulator based on Blender and the Bullet physics engine, designed to remain middleware-agnostic across ROS, YARP, and other frameworks.

- **Habitat** is a 3D simulator for embodied AI research developed by Meta AI Research. It supports training and evaluation of embodied AI agents in photorealistic indoor environments.

- **Flightmare** is a flexible modular quadrotor simulator developed by the Robotics and Perception Group at the University of Zurich and ETH Zurich. It separates rendering from the RL environment for flexible configuration between fast and photorealistic modes.

Drake, MRPT, and SOFA also provide simulation capabilities as part of broader roles (Section 4.6).

4.2.2 Physics Engines and Dynamics Libraries

Physics engines and dynamics libraries provide the computational foundation for simulating rigid-body and multibody dynamics, collision detection, and contact resolution. They often serve as backends to higher-level simulators but can also be used directly. The selected packages primarily classified as physics engines or dynamics libraries are:

- **Bullet/PyBullet** is a widely used open-source physics engine for simulation, robotics, and deep reinforcement learning. PyBullet provides Python bindings that have made it popular in the machine learning community.
- **MuJoCo** (Multi-Joint dynamics with Contact) is a general-purpose physics simulator maintained by Google DeepMind. It is designed for fast and accurate simulation of articulated structures and is widely used in robotics and reinforcement learning research.
- **ODE** (Open Dynamics Engine) is one of the earliest open-source libraries for simulating rigid body dynamics. It has been used extensively in robotics and served as a physics backend for Gazebo Classic.
- **PhysX** is NVIDIA's real-time physics engine SDK. The open-source SDK provides capabilities for simulating rigid bodies, particles, vehicles, and deformable bodies, primarily targeted at real-time simulation applications.

- **DART** (Dynamic Animation and Robotics Toolkit) is a C++ physics engine and kinematics/dynamics library developed at Georgia Tech and other institutions. It is designed for stable simulation in manipulation and locomotion research.

- **Simbody** is a multibody dynamics library developed at Stanford University for simulating articulated biomechanical and mechanical systems. It is the physics engine underlying OpenSim (biomedical simulation software).

- **Pinocchio** is a fast and flexible C++ library for rigid-body dynamics computations and their analytical derivatives, developed primarily at Inria and LAAS-CNRS. It implements state-of-the-art algorithms for kinematics, dynamics, and Jacobians.

- **Project Chrono** is an open-source multi-physics simulation library developed at the University of Wisconsin-Madison and the University of Parma. It supports multibody dynamics, finite element analysis, vehicle dynamics, and GPU-accelerated simulation.

- **Newton Dynamics** is a cross-platform physics simulation engine optimized for real-time deterministic rigid-body simulation. It is used in game development and robotics simulation.

- **SOFA** (Simulation Open Framework Architecture) is an open-source real-time simulation framework developed primarily at Inria. It focuses on deformable object simulation, finite element methods, and haptic feedback, with applications in surgical simulation and soft robotics.

4.2.3 Motion Planning, Middleware, and Manipulation Packages

Motion planning libraries, robotics middleware, and manipulation packages cover parts of the workflow outside full-world simulation: planning algorithms, communication layers, grasp planning, and system integration. They are included because MPS workflows typically combine planning, simulation, and execution rather than using simulators alone. The selected packages primarily classified as planning or middleware packages are:

- **ROS 2** (Robot Operating System 2) is an open-source software development kit and middleware framework for building robot applications. It provides communication infrastructure (based on DDS), hardware abstraction, drivers, libraries, visualization tools, and a large ecosystem of community packages. ROS 2 is the successor to ROS 1 and represents a major redesign with emphasis on real-time control, multi-robot deployment, and production use.

- **MoveIt!** refers to the active MoveIt 2 generation for ROS 2 in this report. It is an open-source robotics manipulation framework that provides motion planning, kinematics, collision checking, trajectory execution, and perception integration for robot arms and manipulators, integrating OMPL and other planners as plugins.

- **OMPL** (Open Motion Planning Library) is an open-source C++ library for sampling-based motion planning algorithms developed at Rice University. It provides implementations of widely used planners such as PRM, RRT, RRT*, and EST, and is the default planning backend for MoveIt!.

- **OpenRAVE** (Open Robotics Automation Virtual Environment) is an open-source robotics planning framework focused on simulation and analysis of kinematic and geometric information for motion planning, particularly for manipulation tasks.

- **Drake** is an open-source C++ and Python toolbox for model-based design and verification of robotics systems, developed by MIT and the Toyota Research Institute (TRI). It covers multibody dynamics, trajectory optimization, model predictive control, perception, and manipulation planning.

- **OROCOS** (Open RObot COntrol Software) is an open-source C++ framework for real-time robot control developed at KU Leuven. Its major components include the Real-Time Toolkit for component-based hard real-time software and the Kinematics and Dynamics Library.

- **MRPT** (Mobile Robot Programming Toolkit) is an open-source cross-platform C++ library for mobile robotics research, providing algorithms for SLAM, computer vision, motion planning, and sensor data processing.

- **YARP** (Yet Another Robot Platform) is open-source middleware for humanoid and general robotics, developed at the Istituto Italiano di Tecnologia. It provides communication infrastructure for distributed robot systems and is the primary middleware for the iCub humanoid robot platform.

- **OpenRTM-aist** is an implementation of the OMG-standardized Robot Technology Component standard, developed at AIST in Japan. It provides a component-based framework for creating and connecting robot software modules.

- **GraspIt!** is a simulation environment for robotic grasping research developed at Columbia University. It supports loading robot hand models and 3D object models to plan, evaluate, and visualize grasps.

4.3 Commonalities

Despite the diversity of the selected packages, several capabilities and concerns recur across the software family. These commonalities are organized around the abstraction of a typical robotics software workflow: model representation, computation, interaction, and documentation. Figure 4.1 presents the family as a commonality/variability feature model: the recurring common concerns (C1–C7) are detailed in this section, and the variable features (V1–V9) in Section 4.4.

C1: Robot Model Representation. All selected packages provide or consume some form of robot model representation. Simulators and physics engines represent robot kinematics and dynamics through internal model structures (often supporting standard formats such as URDF, SDF, or MJCF). Motion planning libraries operate on kinematic and geometric descriptions of robots and environments. Middleware packages define standardized message formats for communicating robot state.

C2: Physics or Kinematics Computation. The packages in the simulator and physics engine roles share the capability of computing robot motion under physical constraints. This includes forward and inverse kinematics, rigid-body dynamics, collision detection, and contact resolution. Even middleware and planning packages typically include or depend on kinematic computation.

C3: Programmatic Interface. All 28 packages provide a programmatic interface (typically a C++ API, often with Python bindings) that allows developers to

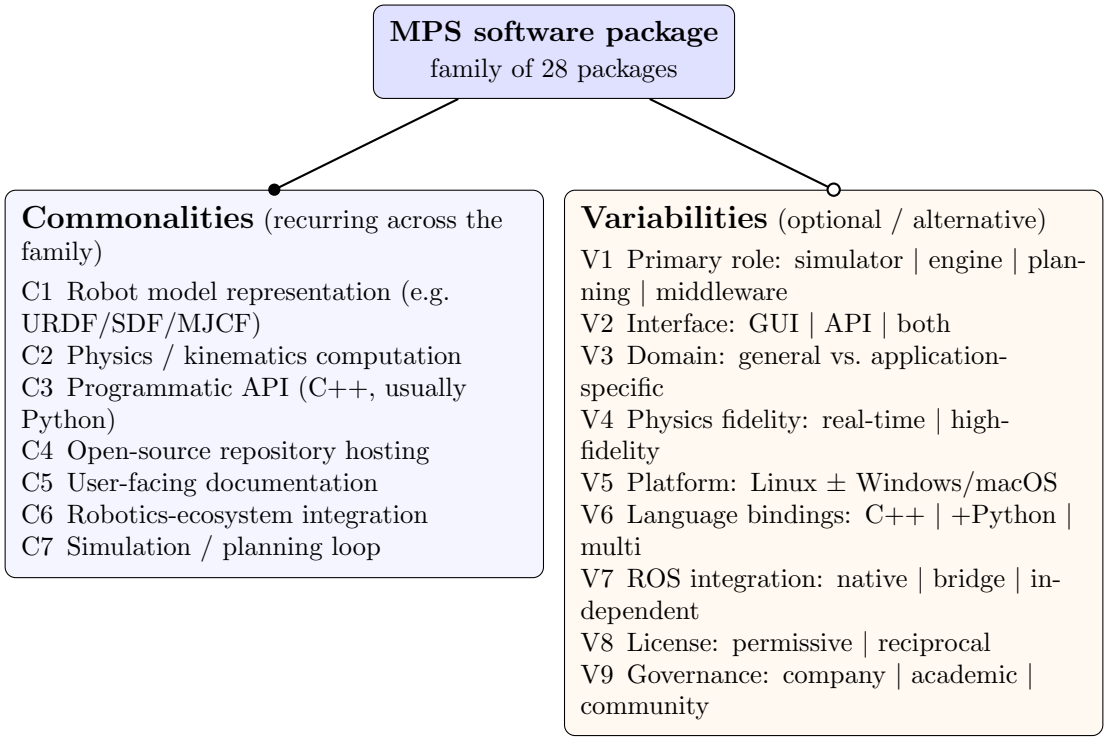

Figure 4.1: Commonality/variability feature model for the 28-package MPS family. Filled circle (•), commonality; open circle (◦), variability.

control the software, configure simulations or planners, and integrate the package into larger robotics workflows.

C4: Open-Source Repository Hosting. All 28 packages use public version control hosting (GitHub for 27 of 28 packages) with visible commit history, issue tracking, and pull request management. This shared infrastructure is both a selection criterion and a commonality of the selected set.

C5: Documentation. All packages provide some form of user-facing documentation, typically including installation instructions, a getting-started tutorial, and API reference material. The depth, currency, and completeness of this documentation vary considerably, as discussed in Section 5.6.

C6: Integration with Robotics Ecosystems. Most packages provide or support integration with broader robotics ecosystems. ROS 2 integration is the most common pattern, with many simulators offering ROS 2 interfaces for publishing sensor data and accepting control commands. Some packages (such as Gazebo and MoveIt!) are deeply embedded in the ROS ecosystem, while others maintain their own integration frameworks.

C7: Simulation or Planning Loop. Simulators and planners share a common architectural pattern: a loop that iteratively updates the system state. In simulators, this is the physics stepping loop that advances the simulation in time. In planners, it is the search or optimization loop that explores the configuration space to find a valid path.

Not every commonality holds with equal strength across all 28 packages: C2, C6, and C7 are dominant, role-specific patterns rather than strictly universal properties, and the role-specific exceptions are captured in the variabilities below.

4.4 Variabilities

The selected packages differ along several important dimensions.

V1: Primary Software Role. The most fundamental variability is the role a package plays in a robotics workflow, ranging from full simulation environments to physics engines to planning libraries to middleware frameworks (Section 4.2).

V2: User Interface Model. Some packages provide graphical user interfaces with 3D rendering, scene editors, and interactive controls (Gazebo, Webots, CoppeliaSim). Others are primarily libraries or APIs intended for programmatic use (OMPL, Pinocchio, ODE). Some provide both (Drake, MRPT).

V3: Domain Specificity. Some packages are general-purpose robotics simulators or engines (Gazebo, Bullet, MuJoCo), while others target specific application domains—autonomous driving, aerial vehicles, embodied AI, medical simulation, and grasping (see the role descriptions in Section 4.2).

V4: Physics Fidelity and Approach. Some engines prioritize speed for real-time or reinforcement learning applications (MuJoCo, Bullet), while others emphasize physical accuracy for engineering analysis (Project Chrono, Simbody). Differences include the choice of constraint solver, collision detection algorithm, integration method, and support for deformable bodies or fluid dynamics.

V5: Platform Support. All packages support Linux, and support for other platforms varies. Some packages provide first-class support for Windows and macOS (Webots, CoppeliaSim, Bullet), while others are primarily developed for Linux with limited cross-platform support.

V6: Programming Language Interfaces. C++ is the dominant implementation language across the selected packages, but packages differ in the range and

quality of language bindings they provide. Python bindings are increasingly common because Python has become central to robotics research and machine learning. Fewer packages provide interfaces for languages such as MATLAB, Java, or Rust.

V7: ROS Integration Depth. Some packages (Gazebo, MoveIt!) are tightly integrated and depend on ROS for core functionality. Others (Webots, CARLA) provide ROS bridges as optional components. Some (YARP, OROCOS) are alternatives to ROS that provide their own middleware infrastructure.

V8: License Model. Although all packages are open source, they use different licenses. The most common are BSD (2-clause and 3-clause), Apache 2.0, MIT, and LGPL. License choice affects how the software can be reused and integrated into other projects.

V9: Community and Governance. Some packages are stewarded by established organizations (Open Robotics for Gazebo and ROS 2, Google DeepMind for MuJoCo, NVIDIA for PhysX), some by academic research groups (Drake at MIT/TRI, OMPL at Rice, DART at Georgia Tech), and some by individual maintainers or small communities (ODE, Newton Dynamics, GraspIt!).

Spatial dimensionality was considered as a possible variability but is not listed among V1–V9: every selected package operates in three dimensions. The one 2D-capable candidate, Player/Stage, was not selected (Appendix C).

4.5 Parameters of Variation

For each variability identified in Section 4.4, Table 4.1 summarizes the parameters of variation and their typical binding times (defined in Section 2.5).

The binding times reflect when a user or integrator can still make a choice: roles

Table 4.1: Parameters of variation and binding times.

Variability	**Parameters of Variation**	**Binding Time**
V1: Primary role	Simulator, physics engine, dynamics library, motion planning library, manipulation framework, middleware, multi-purpose toolbox	Specification time
V2: User interface	Graphical (3D scene, editor), programmatic (API only), both	Specification time
V3: Domain specificity	General-purpose robotics, autonomous driving, aerial vehicles, embodied AI, medical/soft robotics, grasping, mobile robotics	Specification time
V4: Physics fidelity	Real-time (game-quality), high-fidelity (engineering-quality), configurable trade-off	Specification / configuration time
V5: Platform support	Linux-only, Linux + Windows, Linux + Windows + macOS	Specification / build time
V6: Language bindings	C++ only, C++ with Python, multi-language (Python, MATLAB, Java, etc.)	Build / configuration time
V7: ROS integration	Native (ROS-dependent), bridge (optional ROS interface), independent (separate middleware), alternative (non-ROS middleware)	Configuration time
V8: License	Permissive (BSD, MIT, Apache 2.0), reciprocal (LGPL, GPL)	Specification time
V9: Governance	Single institution or company, academic consortium, community-maintained, individual developer	Specification time

and governance are fixed by a package's design, whereas ROS integration and physics fidelity remain configurable later (Table 4.1).

4.6 Boundary Cases

Several of the selected packages occupy intermediate positions in the classification above; the boundaries between roles are not always sharp.

ROS 2 is classified as middleware in this analysis, but its scope extends well beyond communication infrastructure. It is the de facto integration layer for robotics software components, and many other packages in the selected set assume it as their runtime environment.

Drake spans multiple roles. It provides simulation capabilities without being a full simulator in the sense of Gazebo or Webots. It includes sophisticated multibody dynamics without being solely a physics engine. It offers trajectory optimization and model predictive control without being only a planning library.

The boundary between **MoveIt!** and **OMPL** reflects architectural layering rather than functional separation: OMPL is a general-purpose planning library usable independently, while MoveIt! builds kinematics, collision checking, and trajectory execution around it (Section 4.2.3).

SOFA is grouped with the physics engines because the measured materials emphasize deformable-body physics rather than whole-robot simulation (Section 4.2.2); it is a boundary case rather than a direct peer of rigid-body engines such as ODE or Bullet.

In the chapters that follow, measurements are reported for each package individually and interpreted in the context of its role, so these boundary cases are not treated

as measurement anomalies.

Chapter 5

Ranking Projects by Quality Measure

This chapter answers RQ2. It ranks the 28 selected software packages across the nine Domain X quality categories (Section 2.4) and then compares that ranking with community-facing indicators of visibility and adoption. The data were collected in May 2026 and capture the repository state, documentation, and project activity visible at that time. These are public-artifact measurements interpreted under the rules in Chapter 3, not full product evaluations.

5.1 Repository and Project Metadata

Table 5.1 presents summary metadata for all 28 packages, listed alphabetically; platform support is summarized in the narrative below.

Table 5.1: Summary metadata for the 28 selected software packages (May 2026). The Initial Release column records the date associated with the measured public repository: for packages with a single continuous open-source history this is the first public release, while for packages that were renamed, re-licensed, or migrated to open source after an earlier proprietary or pre-release phase (for example CoppeliaSim, MuJoCo, Webots, Gazebo, PhysX), the date reflects that later event rather than the project's original creation.

Software	**Primary Affiliation**	**License**	**Initial Release**	**Primary Languages**
AirSim	Microsoft Research	MIT	2017	C++, Python
Bullet/ PyBullet	Community	zlib	2006	C++, Python
CARLA	CVC Barcelona, Intel	MIT	2017	C++, Python
CoppeliaSim	Coppelia Robotics	GPL + comm.	2019	C++, Python
DART	Georgia Tech	BSD	2011	C++, Python
Drake	TRI, MIT	BSD	2010	C++, Python
Flightmare	U. Zurich, ETH	MIT	2020	C++, Python
Gazebo	Open Robotics	Apache-2.0	2014	C++, Python
GraspIt!	Columbia U.	GPL v3+	2010	C++, Python
Habitat	Meta AI Research	MIT	2019	C++, Python
MORSE	LAAS-CNRS	BSD	2009	C, Python
MoveIt!	PickNik Robotics	BSD	2011	C++, Python
MRPT	U. Almería	BSD	2010	C++, Python
MuJoCo	Google DeepMind	Apache-2.0	2021	C++, Python
Newton Dynamics	Independent	zlib	2013	C++, C
ODE	Independent	BSD/ LGPL	2001	unclear

Software	**Primary Affiliation**	**License**	**Initial Release**	**Primary Languages**
OMPL	Rice University	BSD	2010	C++, Python
OpenRAVE	CMU	LGPL + A2.0	2009	C++, Python
OpenRTM-aist	AIST (Japan)	LGPL-2.1	2005	C++, Python
OROCOS	KU Leuven	GPL + LGPL	2007	C++, Python
PhysX (OSS)	NVIDIA	BSD	2022	C++, Python
Pinocchio	Inria, LAAS-CNRS	BSD	2014	C++, Python
Project Chrono	U. Wisconsin, Parma	BSD	2010	C++, Python
ROS 2	Open Robotics	Apache-2.0	2015	unclear (core C++)
Simbody	Stanford U.	Apache-2.0	2005	C++, C
SOFA	Inria	LGPL-2.1	2005	C++, Python
Webots	Cyberbotics	Apache-2.0	2018	C++, Python
YARP	IIT (Italy)	BSD	2006	C++, Python

All 28 packages have publicly accessible source repositories. Twenty-seven follow an open-source development model under standard permissive or copyleft licenses; CoppeliaSim is the exception, with a mixed model that combines a GPL-licensed open-source codebase with commercial and educational licenses for its packaged distribution. The most common licenses are BSD (11 packages), Apache 2.0 (5), and MIT (4). The packages span more than two decades of development history: ODE is the oldest, with its initial release in 2001, while PhysX's open-source SDK was

released in 2022.

C++ is the dominant implementation language, listed as a primary or major language by 25 of 28 packages in the measurement records. Three packages fall outside that count: MORSE (primary languages: C and Python), ODE (primary language listed as "unclear" because the public source tree mixes C and C++ files without a single declared primary), and ROS 2 (also "unclear" because the ROS 2 organization spans hundreds of repositories with different primary languages, although the core communication libraries themselves are in C++). Python is a primary or secondary language in most packages, reflecting its growing role in robotics research and its use for scripting, bindings, and machine learning integration. All 28 packages support Linux. Twenty-four packages support Windows and 23 support macOS.

As of the May 2026 measurement date, 24 of the 28 packages (86%) were classified as alive under the Domain X 18-month activity rule, which defines alive as the presence of any commit or release in the preceding 18 months. Four packages were classified as dead: OpenRAVE (last commit August 2024), MORSE (last commit March 2022), GraspIt! (last commit July 2020), and Flightmare (last commit May 2023). These four packages were retained in the selected set because they remain prominent in the domain (Section 3.5.3); their dead classification here is a measured outcome rather than a selection criterion. ODE is technically alive under the 18-month rule (last commit 2026-04-14) but its commit activity in the trailing twelve months is sparse (five commits across the year); the analysis that follows treats ODE as a low-activity alive package and discusses its limitations alongside the dead packages where appropriate.

The number of developers (anyone who has made at least one accepted commit)

ranges from 6 (CoppeliaSim, Flightmare) to 598 (MoveIt!). The median is approximately 128.5 (the mean of the 14th and 15th values, YARP at 125 and MRPT at 132). The total number of commits across all measured repositories exceeds 240,000. Drake leads with 34,786 commits, followed by SOFA (27,404) and Project Chrono (19,510). These repository-level metrics are snapshots and should not be interpreted as direct measures of project scale or quality without considering factors such as repository age, commit granularity, and whether development activity is concentrated in a single repository.

PhysX requires a particular caveat. The measured repository (`NVIDIA-Omniverse/PhysX`) is the public open-source SDK release of NVIDIA's PhysX engine; the broader production engine and its associated engineering process are developed in an internal NVIDIA codebase that is not publicly accessible. All PhysX scores reported in this chapter, including the small public commit count and the per-quality impression scores, reflect only the public SDK surface as it appeared at the measurement date. They should not be read as an assessment of the PhysX engine as a product, or of NVIDIA's internal development practices.

Two further repository-identity caveats apply. The OROCOS measurements are based on the `orocos_kinematics_dynamics` repository, which hosts the Kinematics and Dynamics Library (KDL); the project's other components, such as the Real-Time Toolkit, live in separate repositories and are not represented in the OROCOS code-size, commit, or activity figures. For Newton Dynamics, the measured repository is the maintainer's current development home, created in January 2026 when development moved away from the project's long-standing original repository. Its star and fork counts therefore describe a repository that was only a few months old

at measurement time, not the community attention the project accumulated over two decades.

5.2 Installability

Installability (Section 2.4) was assessed on Linux-based virtual machines (Section 3.7.3). Table 5.2 summarizes key installability indicators across the selected packages.

Table 5.2: Key installability indicators across 28 packages.

Indicator	**Packages**	**%**
Installation instructions present	28	100%
Instructions in a single document or page	26	93%
Instructions are linear (single series of steps)	26	93%
Instructions assume no pre-installed dependencies	22	79%
Compatible OS versions listed	17	61%
Installation automation provided (script, makefile, etc.)	28	100%
Installation validation procedure available	27	96%
Dependency installation instructions provided	21	75%
Required package versions listed	19	68%
No problems caused by uninstall (where available)	27	96%

All 28 packages provide installation instructions and some form of installation automation (makefiles, scripts, package managers, or container definitions).

The number of installation steps ranged from 1 to 11, with a median of 2. The number of extra software packages required before or during installation ranged from

0 to 10, with a median of 1. GraspIt! required the most manual effort (11 steps, 10 extra packages). At the other extreme, ten packages (including Webots, MuJoCo, DART, and Simbody) needed no extra packages at all, and nine could be installed in a single step using package managers or pre-built binaries (MuJoCo, Simbody, and MORSE achieved both).

Installation broke for only 2 of the 28 packages during measurement (OpenRAVE and YARP). In both cases the installation instance was recoverable. No package broke during initial tutorial testing. This 2-of-28 breakage rate compares favourably with the Medical Imaging study, where installation instructions were found for only 16 of 29 packages and three packages could not be installed at all (Smith et al., 2024a).

The mean installability score was 7.4 (range 4–10). The low scores reflect documentation gaps: ODE's official installation guide dates from 2006 and refers to GNU make, VC6 project files, and manual copying of library files, while PhysX's correct build documentation is hard to locate. MuJoCo's score of 7, despite a one-step pip installation with no extra packages, reflects a recorded mismatch between installation and verification: running the documented example script after the standard installation failed with a file-not-found error.

5.3 Correctness and Verifiability

Table 5.3 summarizes key correctness and verifiability indicators.

Table 5.3: Correctness and verifiability indicators.

Indicator	**Packages**	**%**
Reference to requirements specifications or theory manuals	26	93%
Getting-started tutorial present	28	100%
Tutorial instructions are linear	27	96%
Tutorial provides expected output	25	89%
Tutorial output matches expected output	23	82%
Unit tests available	27	96%
Evidence that performance was considered	28	100%

All 28 packages use at least one technique intended to support correctness. Table 5.3 records unit tests for 27 of 28 packages (96%). Automated testing is recorded separately: 26 of 28 packages have CI or an equivalent setup that exercises tests or related checks on each push or pull request, giving an automated-testing rate of 93%. Assertions in code are used by 23 of 28 packages (82%). Documentation generation tools such as Doxygen or Sphinx are used by 12 packages (43%). Ten packages combine all three techniques: DART, Drake, Habitat, Newton Dynamics, OMPL, OROCOS, PhysX, Pinocchio, Project Chrono, and SOFA all use automated testing, assertions, and a documentation generator together.

The tutorial-output check succeeded for 23 of the full 28-package set (82%). Five packages fall outside that positive count: SOFA, Flightmare, and Newton Dynamics do not publish an expected tutorial output, while MuJoCo and OpenRTM-aist had environment-dependent outputs that the standard measurement procedure could not reproduce.

CoppeliaSim is the one package without visible unit tests: its status is unclear because its test infrastructure is not publicly visible in the same way as the other packages, a consequence of its mixed commercial/open-source licensing model.

Correctness and verifiability averaged 7.7 (5–10), with GraspIt! and Newton Dynamics (5) lowest.

5.4 Surface Reliability

Surface reliability covers only the interactions performed during measurement: installation and initial tutorial use.

The two installation breakages noted in Section 5.2 (OpenRAVE, YARP) feed surface reliability: for OpenRAVE a descriptive error message was displayed, while the record does not capture YARP's error output.

Surface reliability had a mean of 7.8 (6–10); its leaders mirror the correctness ranking.

5.5 Surface Robustness

Surface robustness was assessed for all 28 packages as handling unexpected input reasonably (producing appropriate error messages or graceful degradation rather than crashes or undefined behaviour). For the newline-conversion check (replacing newlines with carriage returns and newlines in plain-text input files), 25 of 28 packages handled the conversion gracefully; the remaining three (CoppeliaSim, ODE, PhysX) were recorded as not applicable because their workflows do not take plain-text input files in this form.

For surface robustness the mean was 7.4 (5–10). Because both binary checks pass for nearly all packages, the score spread comes from the broader observations feeding the impression calculator: the quality of error messages and input validation encountered during installation and tutorial use (Section 3.7.1).

5.6 Surface Usability

Surface usability covers user-facing documentation, tutorials, and support infrastructure; no user studies or task-completion experiments were conducted.

All 28 packages provide a getting-started tutorial and a user manual. Only one package in the current measurement (MuJoCo) documents expected user characteristics; the LBM study similarly reported this artifact as rare (Smith et al., 2024b), while the MI study does not report it as a count (Smith et al., 2024a).

Table 5.4 summarizes the user support models adopted by the study packages; most use multiple channels. A support route is counted only when the measurement record treats it as current user or developer support, so historical channels, commercial-inquiry contacts, and indirectly affiliated community spaces are excluded from the totals.

Table 5.4: User support models across 28 packages.

Support Channel	**Packages Using**
GitHub Issues / equivalent issue tracker	28
GitHub Discussions	10
Official forum or discussion board	9
Mailing list / Google Group	8
Discord server	6
Documentation website or wiki	28
Stack Exchange / Stack Overflow tag	5
ROS Answers integration	4

Beyond the universal issue tracker and documentation website, the pattern partly tracks project age and governance model. Newer packages and those stewarded by companies or large labs tend to favour Discord and GitHub Discussions, while older academic packages more commonly use mailing lists and dedicated forums.

Surface usability averaged 7.3 (5–10); its top scorers pair multiple support channels with thorough user, developer, and API documentation.

5.7 Maintainability

Table 5.5 summarizes key maintainability indicators.

Table 5.5: Maintainability indicators across 28 packages.

Indicator	**Count**	**%**
Version number documented	28	100%
Code review / contribution guide	26	93%
Non-code artifacts present	28	100%
Issue tracking via GitHub or equivalent	28	100%
Version control via git (GitHub or BitBucket)	28	100%

All 28 packages use git for version control. Twenty-seven of the 28 host their primary repository on GitHub; ODE is the exception, hosting on BitBucket alongside a GitHub mirror. All 28 use GitHub Issues or the equivalent (BitBucket Issues for ODE) for issue tracking. This uniformity is partly a selection effect, since the Domain X methodology expresses a preference for GitHub-style repositories because they facilitate consistent measurement; it also reflects the near-universal adoption of git and GitHub in the contemporary robotics software community.

The percentage of identified issues that are closed ranges from 34.6% (Flightmare) to 100% (DART, Newton Dynamics), with a mean of 79.1% and a median of 82.0% across the 27 packages that yielded a measurable closure rate (ODE returns no value because the BitBucket API call did not populate this field, as noted under Table 5.8). A high closure rate can indicate active issue management, but it can also reflect a low rate of new issue reporting or a practice of closing stale issues aggressively.

The percentage of code that consists of comments ranges from 1.1% (Project Chrono) to 28.0% (OROCOS), with a mean of 13.2% across the 27 packages with comparable single-repository measurements. The ROS 2 entry is excluded from this

range and mean because its multi-repository structure does not yield a comparable single-repository code-line count (Table 5.8 marks it as not measured rather than as zero). The Domain X methodology treats 10% as a threshold for the maintainability impression calculator.

The mean maintainability score was 7.5 (4–10); GraspIt! and Flightmare (4) ranked lowest, held back by their dead or very low activity status and limited contribution infrastructure.

5.8 Reusability

Reusability is measured through two indicators: the number of code files (a rough proxy for component granularity) and the presence of API documentation.

The number of code files varies widely across the 27 packages with a comparable single-repository count, from 118 (Flightmare) to 7,823 (Newton Dynamics), with a median of 1,266 (Pinocchio at the midpoint of the sorted list). ROS 2 is again excluded, for the multi-repository reason noted in Section 5.7. All 28 packages provide API documentation; where the measurement records identify documentation-generation tools, the most common are Doxygen and Sphinx.

Reusability had a mean of 7.6 (5–10); its highest scorers are built for integration into larger robotics systems or serve as a foundation for other tools.

5.9 Surface Understandability

Surface understandability was assessed through a review of 10 randomly selected source files per package. One template question, whether parameters appear in the

same order across functions, was not recorded during measurement; the discussion and scores below rest on the remaining checks.

Table 5.6 summarizes the surface understandability indicators.

Table 5.6: Surface understandability indicators across 28 packages.

Indicator	**Packages**	**%**
Consistent indentation and formatting style	28	100%
Explicit identification of a coding standard	14	50%
Consistent, distinctive, and meaningful identifiers	28	100%
Hard-coded constants (other than 0 and 1) present	28	100%
Comments are clear and describe what is being done	28	100%
Name/URL of algorithms used is mentioned	28	100%
Code is modularized	28	100%

All 28 packages exhibit consistent indentation and formatting, meaningful identifiers, clear comments, and modularized code organization. These are baseline indicators of professional software development practice, and their universal presence is a positive signal for the MPS domain.

Fourteen of 28 packages (50%) explicitly identify a coding standard. This is stronger than the MI comparison point, where code style guides were rare (Smith et al., 2024a), but it still leaves half of the measured MPS packages without an explicit coding-standard artifact. Packages that do identify a coding standard include Gazebo, Webots, Bullet/PyBullet, MuJoCo, ROS 2, Drake, OMPL, Pinocchio, MoveIt!, Habitat, Project Chrono, MRPT, YARP, and OpenRTM-aist.

Surface understandability averaged 7.3 (5–10).

5.10 Visibility and Transparency

Table 5.7 summarizes the key visibility and transparency indicators.

Table 5.7: Visibility and transparency indicators.

Indicator	**Packages**	**%**
Development process defined (CI, contributing guide, etc.)	25	89%
Documents recording development process and status	28	100%
Development environment documented	28	100%
Release notes published	27	96%

A defined development process (25 of 28 packages) is typically manifested through CI/CD workflows, contributing guidelines, pull request templates, and issue templates.

Three packages are recorded as unclear for the defined-development-process metric: CoppeliaSim, ODE, and OMPL. For CoppeliaSim, the mixed licensing model means parts of the development process are not publicly visible. For ODE, the unclear result is consistent with its low activity level and sparse process documentation. For OMPL, the result is best read narrowly as the outcome of this measurement item, not as a claim that the project lacks public development infrastructure in every respect.

The mean visibility and transparency score was 7.4 (5–10); the leaders publish thorough documentation of their development workflows.

5.11 Repository Metrics

Table 5.8 presents key repository metrics for all 28 packages (Section 3.7.2), including GitHub popularity indicators and codebase size measures.

Notes: ROS 2 code line and comment counts are not directly comparable because the ROS 2 organization spans hundreds of repositories. ODE is hosted on BitBucket; star, fork and issue-closure metrics are reported as "—" because the BitBucket APIs used during measurement did not return values for these fields. The "code lines" column reports SCC-measured code lines in the primary measured repository. † PhysX and ‡ Newton Dynamics carry the repository-identity caveats of Section 5.1 (public SDK only; post-migration repository), as does the OROCOS (KDL) row; Newton Dynamics' star and fork counts exclude the project's original repository (roughly 1,000 stars as of June 2026).

GitHub stars (a rough proxy for community attention) span almost three orders of magnitude, from 24 (OpenRTM-aist, and Newton Dynamics with the repository-migration caveat noted under Table 5.8) to 18,159 (AirSim). The median among the 27 packages with GitHub star data is approximately 2,050. The correlation between stars and measurement scores is not straightforward. AirSim leads in stars by a wide margin but does not appear among the top-scoring packages in most quality categories. Conversely, Gazebo scores highly across most quality categories but has relatively modest star counts (1,337), likely because its community engagement predates the current GitHub repository organization and because its user base interacts with it primarily through ROS rather than directly through its GitHub page.

The number of commits ranges from 40 (PhysX) to 34,786 (Drake). If PhysX is excluded as a special public-SDK mirror of a broader internal codebase, the smallest

Table 5.8: Repository metrics for the 28 selected packages (May 2026), listed alphabetically.

Software	Stars	Forks	Commits	Code Lines	% Comments	% Issues Closed
AirSim	18,159	4,889	3,562	134,331	10.3%	80.0%
Bullet/ PyBullet	14,465	3,074	9,405	1,528,236	2.8%	87.3%
CARLA	13,941	4,559	6,713	138,801	9.4%	82.0%
CoppeliaSim	146	46	1,206	292,460	5.1%	95.5%
DART	1,081	300	6,727	339,389	9.2%	100.0%
Drake	4,025	1,366	34,786	705,686	20.5%	90.0%
Flightmare	1,353	400	139	14,543	20.1%	34.6%
Gazebo	1,337	411	7,500	193,808	11.9%	56.7%
GraspIt!	213	86	733	110,614	2.1%	56.6%
Habitat	3,662	530	1,664	143,150	7.7%	75.7%
MORSE	367	156	4,356	141,241	2.7%	77.6%
MoveIt!	1,803	744	9,462	159,789	26.1%	80.5%
MRPT	2,133	658	10,231	504,916	9.4%	95.4%
MuJoCo	13,440	1,501	4,676	349,490	6.7%	91.7%
Newton Dynamics	24‡	5‡	17,755	1,585,430	15.7%	100.0%
ODE	—	—	2,119	147,055	18.8%	—
OMPL	2,047	690	5,341	105,532	27.4%	93.2%
OpenRAVE	801	355	10,375	623,306	19.2%	42.9%
OpenRTM-aist	24	14	4,360	86,965	26.3%	81.4%
OROCOS	879	443	1,229	18,063	28.0%	78.0%
PhysX (OSS)	4,532	614	40†	1,030,524	17.3%	67.9%
Pinocchio	3,354	537	13,081	185,472	11.9%	92.3%
Project Chrono	2,836	595	19,510	657,303	1.1%	96.8%
ROS 2	5,475	903	309	—	—	86.2%
Simbody	2,521	492	5,041	259,447	25.3%	59.5%
SOFA	1,194	352	27,404	349,961	4.5%	64.1%
Webots	4,345	2,025	15,746	581,442	1.9%	87.9%
YARP	590	214	19,020	843,982	15.1%	82.3%

measured commit count is 139 (Flightmare, which is now inactive). Commit activity in the last 12 months shows a similar range: the most active projects (such as Newton Dynamics, Pinocchio, MuJoCo, and Drake) sustain high monthly activity, while several dead packages (MORSE, GraspIt!, Flightmare) show zero or near-zero recent commits.

Against the 10% comment-density threshold used by the maintainability calculator (Section 5.7), 15 of the 27 packages with comparable measurements (about 56%) exceed it. Some of the largest codebases in the study fall below the threshold, so codebase size alone does not determine comment density.

5.12 Overall Observations

Table 5.9 summarizes the overall impression scores across all nine quality categories for each package, along with a simple unweighted per-package mean. Figure 5.1 plots the per-package mean as a ranked bar chart coloured by activity status, Figure 5.2 renders the full score matrix as a heatmap, and Figure 5.3 shows how the scores are distributed within each of the nine categories. The scores reported here are the raw 1–10 impression values assigned during measurement using the Domain X Appendix C calculator. The unweighted mean treats all nine qualities as equally important. Figure 5.4 presents the illustrative equal-weight AHP ranking specified in Section 3.7.4; under equal weights the AHP global priorities and the unweighted mean agree at the top of the table.

A handful of packages anchor the top of the table. Gazebo scores 10 in all nine categories, the only such sweep in the study. Within this template and time budget, no measured indicator separated it from the top of each scale; this is not a claim

Table 5.9: Overall impression scores by quality category (1–10 scale) with an unweighted per-package mean. I = Installability, C = Correctness, SR = Surface Reliability, SRo = Surface Robustness, SU = Surface Usability, M = Maintainability, Re = Reusability, SUnd = Surface Understandability, VT = Visibility/Transparency. The Mean column is the simple arithmetic mean of the nine category scores.

Software	**I**	**C**	**SR**	**SRo**	**SU**	**M**	**Re**	**SUnd**	**VT**	**Mean**
AirSim	8	8	8	8	8	8	8	7	7	7.8
Bullet/ PyBullet	9	7	8	7	7	7	8	7	7	7.4
CARLA	7	8	8	7	8	8	8	7	8	7.7
CoppeliaSim	8	8	8	7	7	7	8	7	7	7.4
DART	8	8	8	7	7	8	8	7	7	7.6
Drake	7	10	10	9	9	9	8	8	9	8.8
Flightmare	7	6	7	8	6	4	6	6	5	6.1
Gazebo	10	10	10	10	10	10	10	10	10	10.0
GraspIt!	5	5	6	5	5	4	5	5	5	5.0
Habitat	8	8	8	8	8	8	8	8	8	8.0
MORSE	9	8	8	8	8	8	5	6	5	7.2
MoveIt!	7	9	9	8	9	9	9	9	9	8.7
MRPT	8	8	7	8	7	8	8	8	8	7.8
MuJoCo	7	7	8	8	8	8	9	8	8	7.9
Newton Dynamics	6	5	6	6	5	5	6	6	5	5.6
ODE	4	6	7	7	6	6	7	7	7	6.3
OMPL	9	9	9	8	8	8	9	9	9	8.7
OpenRAVE	8	8	8	7	8	8	7	8	7	7.7
OpenRTM-aist	7	8	7	7	6	7	6	7	7	6.9
OROCOS	7	8	7	6	6	6	7	6	6	6.6
PhysX (OSS)	4	6	6	6	7	8	8	8	8	6.8
Pinocchio	10	9	9	8	8	9	9	8	8	8.7
Project Chrono	7	8	8	7	7	8	7	7	8	7.4
ROS 2	8	9	9	8	9	9	9	8	9	8.7
Simbody	7	7	7	7	7	7	7	7	7	7.0
SOFA	6	6	8	7	6	7	7	7	7	6.8
Webots	8	8	8	7	8	8	8	7	8	7.8
YARP	7	8	7	7	7	8	7	7	8	7.3

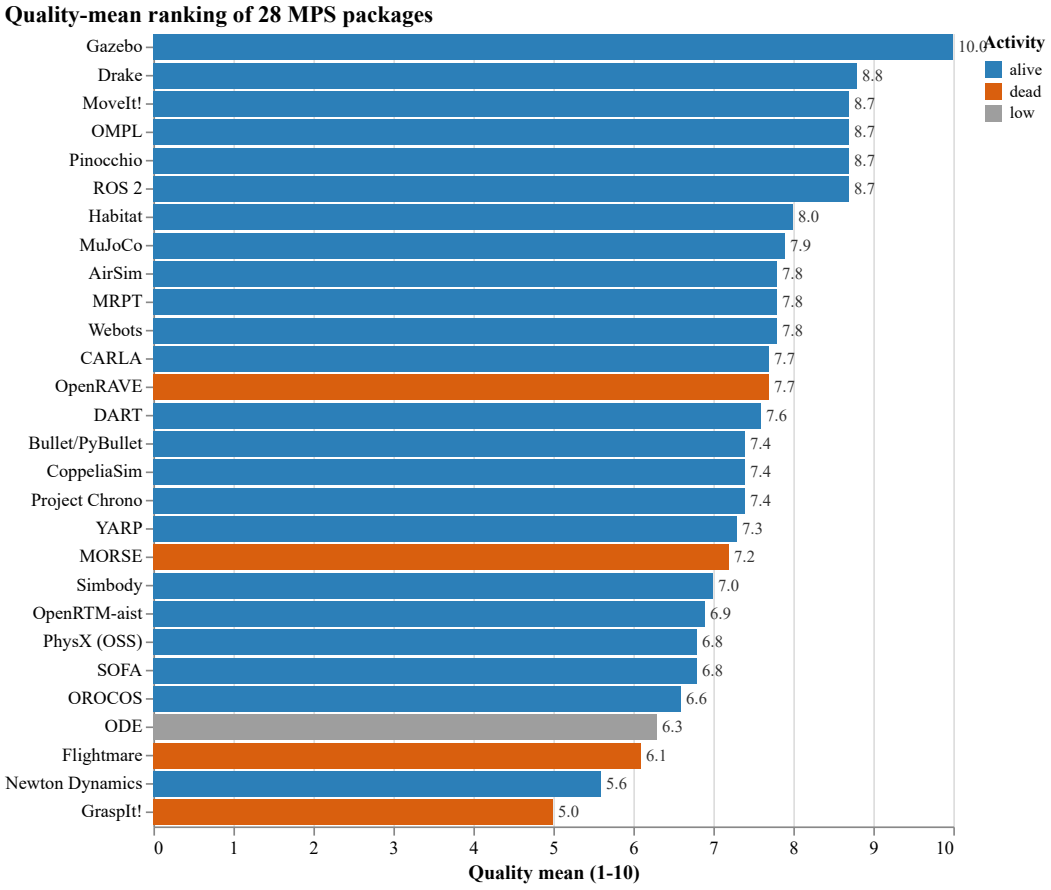

Figure 5.1: Quality-mean ranking of the 28 packages (unweighted mean of the nine category scores), coloured by activity status.

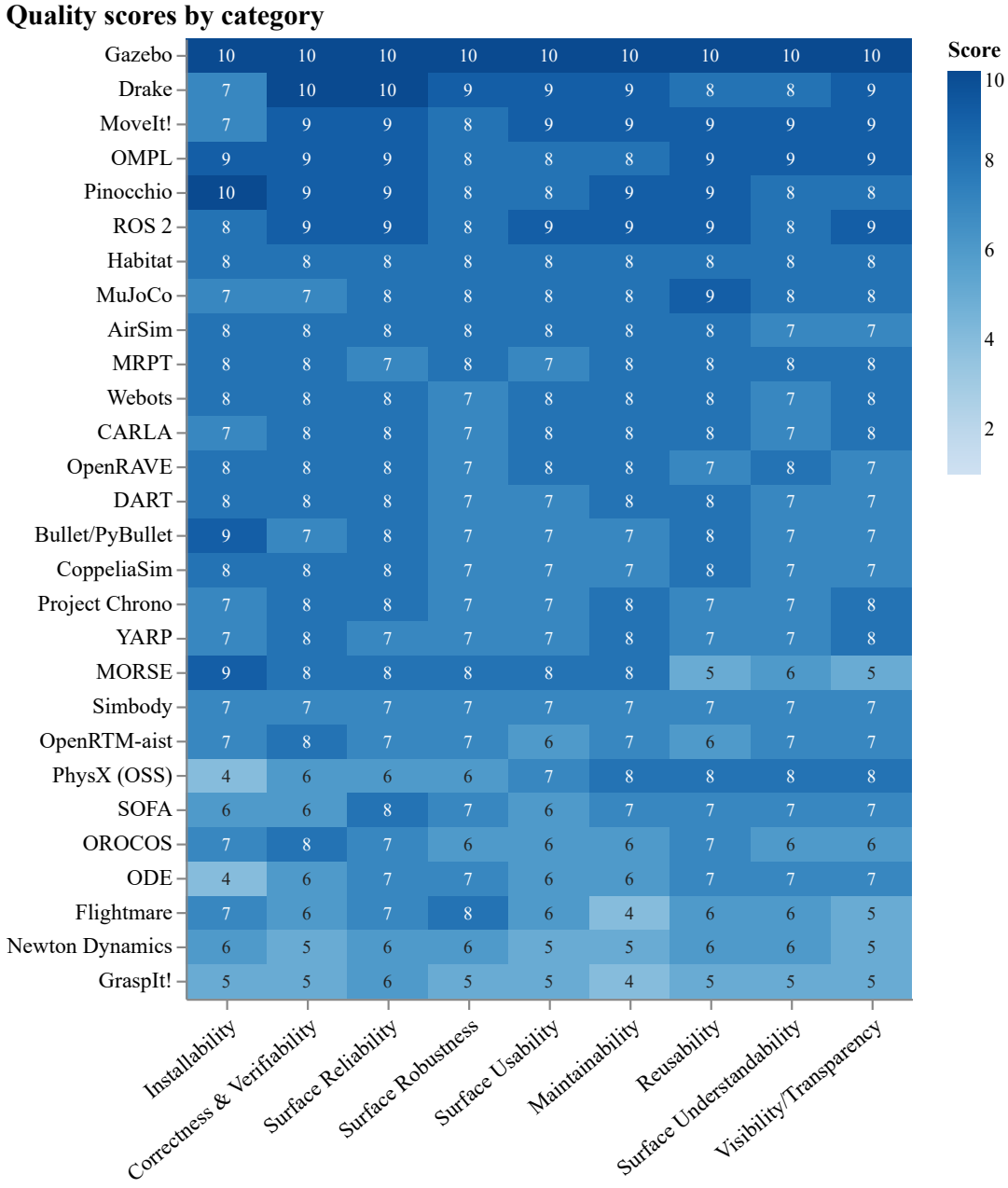

Figure 5.2: Heatmap of the quality impression scores (1–10) for the 28 packages across the nine quality categories, ordered by unweighted mean.

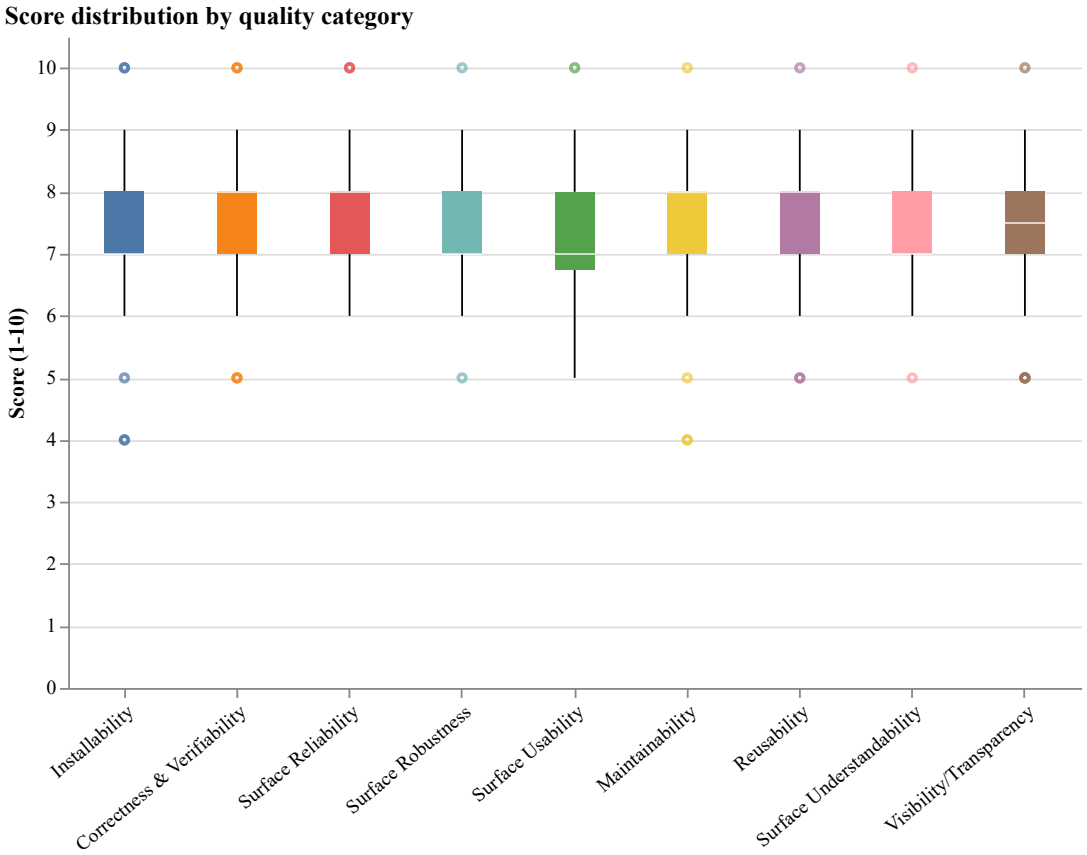

Figure 5.3: Distribution of the impression scores across the 28 packages within each of the nine quality categories (box plots).

that the software is flawless. Drake and MoveIt! score 9 or 10 across most categories, and ROS 2, OMPL, and Pinocchio sit just below them. These packages share several traits: established stewardship (large organizations or active research groups), continuous contributor activity, comprehensive documentation, automated testing wired into CI/CD, and published contribution guidelines and release notes. None of these traits is individually decisive, but the combination separates the top tier from the rest.

For comparability with the descriptive summaries reported in the LBM and Medical Imaging assessments (Smith et al., 2024b,a), Table 5.9 was also examined using a top-two coverage rule. A package was counted when it occupied one of the two highest distinct impression-score tiers in at least two of the nine quality categories. Six of the 28 packages (21%) meet this criterion: Gazebo, Drake, MoveIt!, OMPL, Pinocchio, and ROS 2. The corresponding studies reported 67% for LBM and almost 70% for Medical Imaging. This difference does not establish that the MPS domain is unhealthy: the statistic is sensitive to tied scores, score calibration, the qualities measured, and the package-selection process in each study. Here it is used only to characterize the concentration of top-tier performance, which is clustered in a small leading group rather than broadly distributed across the sample.

To check that this ordering does not hinge on the precise impression scores, the AHP ranking was recomputed under a Monte Carlo perturbation of its inputs. Each of the 252 score entries in Table 5.9 was independently scaled by a factor drawn uniformly from $[0.9, 1.1]$ (a $\pm 10\%$ perturbation clipped to the 1–10 range), and the equal-weight global priorities were re-derived over 1,000 trials with a fixed random seed.

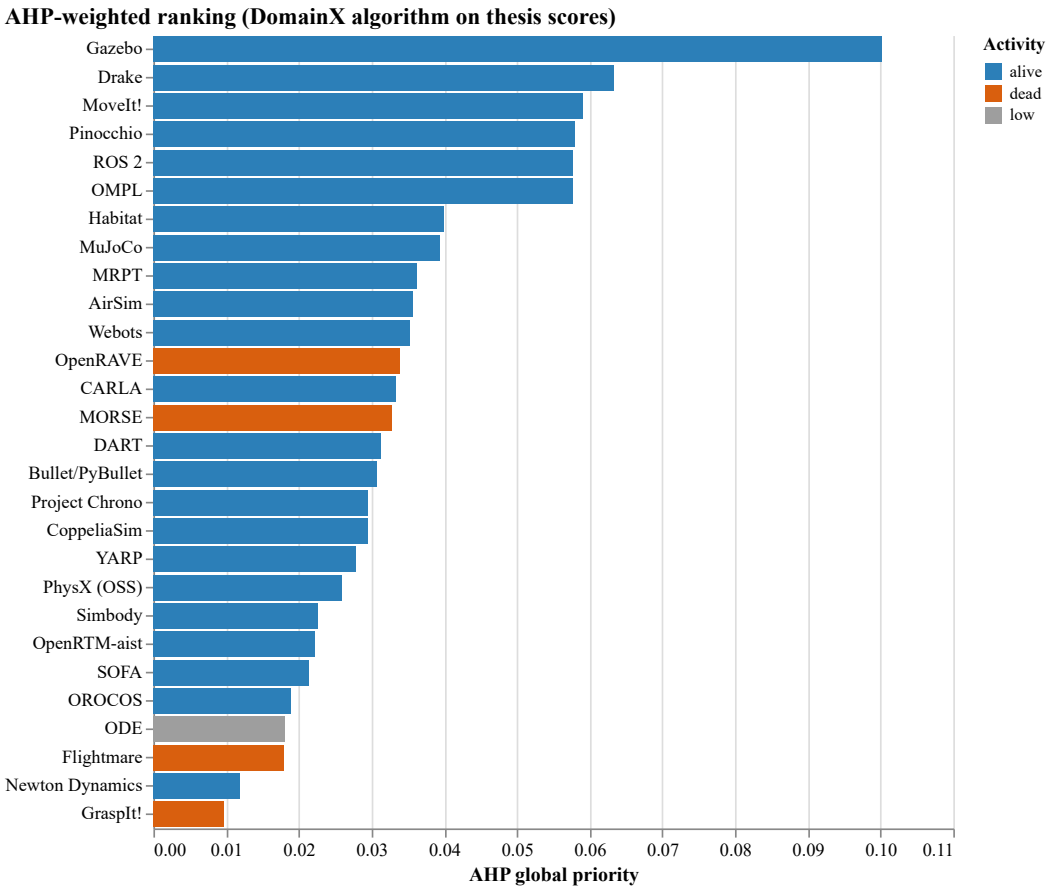

Figure 5.4: Illustrative equal-weight AHP global-priority ranking ($c_k = 1/9$) of the 28 packages.

The ordering is well preserved: the mean Spearman rank correlation with the baseline is 0.97 (minimum 0.87) and the mean Kendall τ is 0.88 (minimum 0.74). Gazebo remains top-ranked in every trial, and the two lowest, Newton Dynamics and GraspIt!, are likewise fixed. The five highest-scoring packages after Gazebo never leave the top eight positions, though they are interchangeable within that band, with Drake alone ranging over ranks 2–6 (Figure 5.5).

The exact ordering among the leaders should therefore not be over-interpreted: the precise top-three and top-five sets recur in only 22% and 26% of trials, because their score gaps are smaller than the perturbation. The largest movement, up to ten positions, occurs in the densely packed middle of the table. The check thus supports the coarse structure of the ranking: a clear leader, a stable leading tier, a closely packed middle, and a fixed bottom (Section 2.6).

The bottom of the table tracks maintenance status, but only loosely. GraspIt!, Flightmare, and MORSE (the dead packages of Section 5.1) appear in the lower tier for most quality categories. Newton Dynamics, though technically alive at the May 2026 measurement date, joins them, reflecting its largely individual-maintained codebase and thin documentation infrastructure. OpenRAVE is also classified as dead under the 18-month rule, but its quality scores cluster in the middle of the table rather than the bottom; the documentation, test infrastructure, and codebase organization built during years of active development have not visibly degraded in the period since. In short, packages that go inactive tend to accumulate outdated documentation, broken installation procedures, and unresolved issues, but the rate at which they do so depends on the engineering capital accumulated before they went quiet.

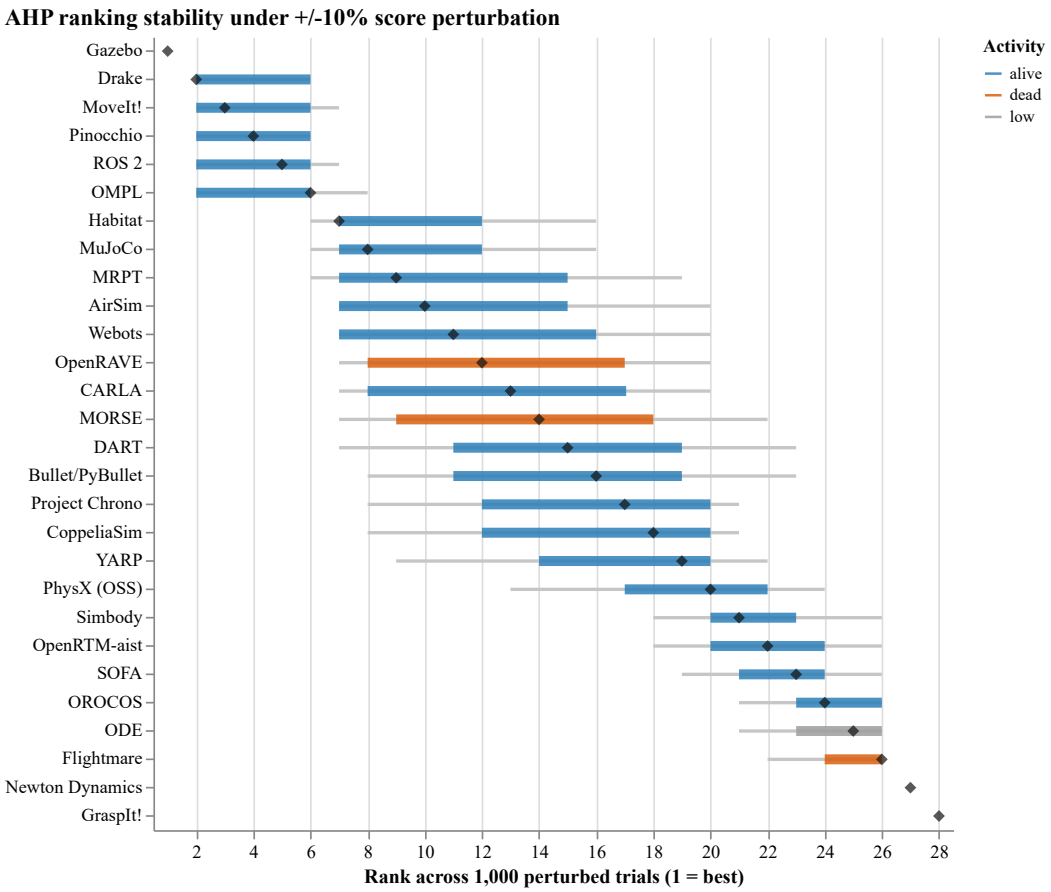

Figure 5.5: Stability of the equal-weight AHP ranking under a ±10% score perturbation. Diamond, baseline rank; thick bar, 5th–95th percentile; thin line, full range.

Installability is generally strong across the MPS domain, most plausibly because of the widespread adoption of standard package managers (APT, pip, Conda), containerization (Docker), and CMake-based builds in the robotics community.

Documentation is universally present but varies in depth. All packages provide tutorials, user manuals, and API documentation, but the two indicators that record explicit engineering intent, coding standards and documented user characteristics, are far less common. The same gaps appear in the medical imaging and LBM samples (Smith et al., 2024a,b), so they look more like patterns of research software practice than peculiarities of MPS. Figure 5.6 summarizes this contrast across the template: the indicators tied to day-to-day installation and use sit at or near full coverage, while coding standards and documented user characteristics fall to 50% and 4%.

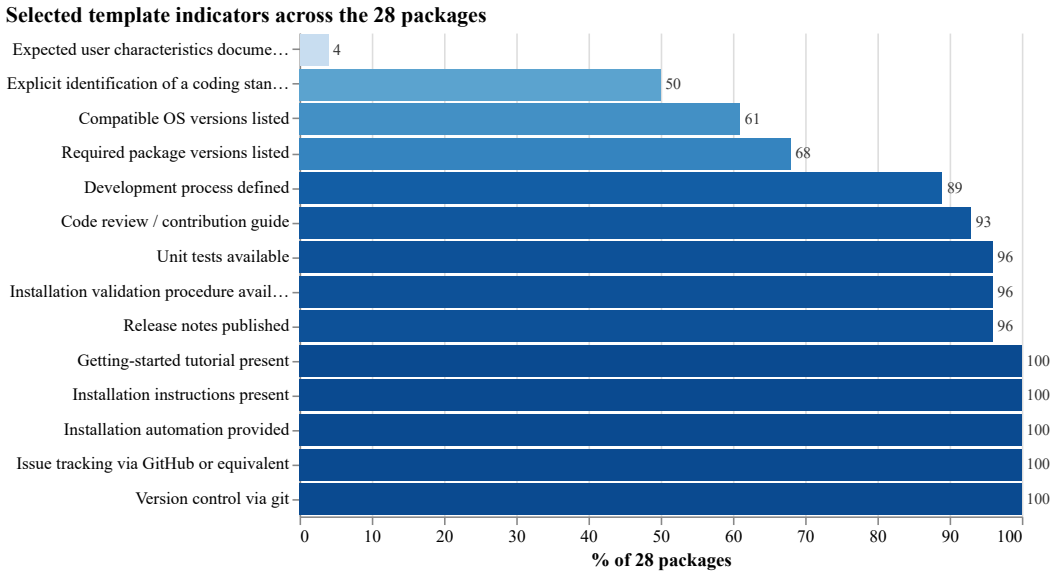

Figure 5.6: Coverage of selected measurement-template indicators across the 28 packages, sorted from least to most common.

Repository activity is concentrated in a small subset of packages, with a few large

or long-running projects accounting for a disproportionate share of total commits while several others show minimal activity in the last 12 months (Section 5.11).

5.13 Comparison with Community Indicators

The Domain X quality mean (Section 5.12) reflects per-package documentation, testing, and release-management practice, while community indicators such as GitHub stars, forks, and recent commit activity capture a different signal: cumulative visibility and adoption built up over time. The two need not agree.

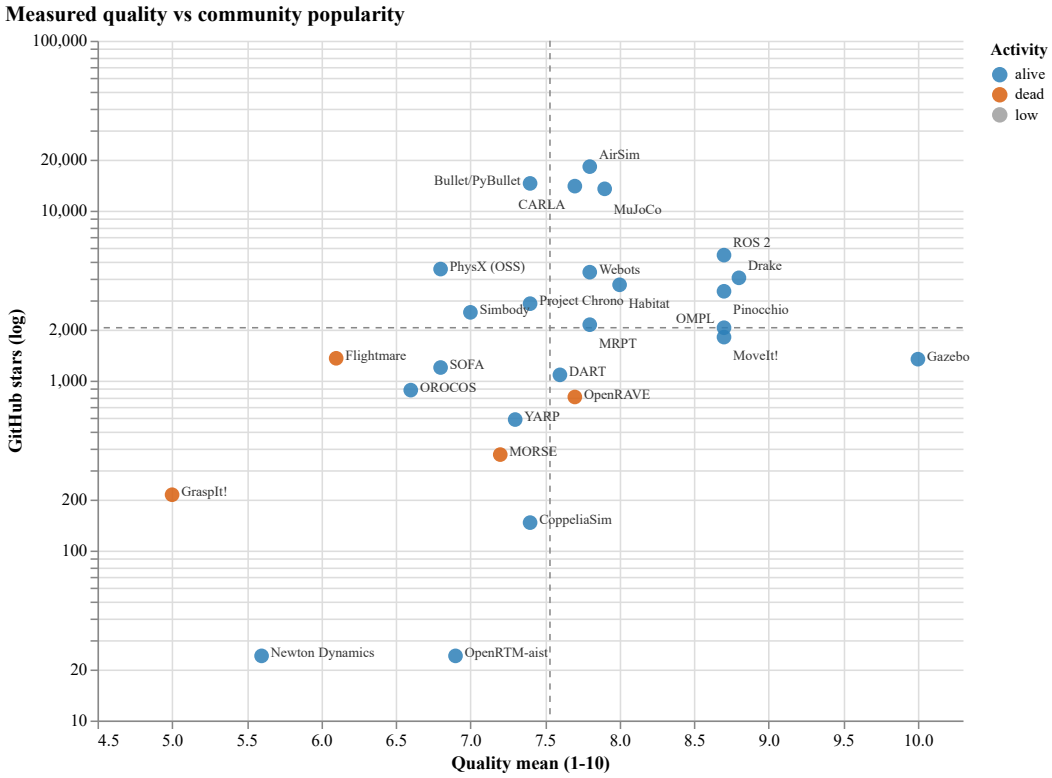

Figure 5.7: Measured quality mean versus GitHub stars (log scale) for the 28 packages, coloured by activity status. Dashed lines mark the mean quality and the median star count.

Table 5.10: Quality-mean rank versus GitHub stars rank for the 28 selected packages. $\Delta = S - Q$: positive values indicate a higher quality rank than stars rank (under-recognized for its quality); negative values indicate a higher stars rank than quality rank (popular without proportional quality scores). Ties receive the same rank. ODE is hosted on BitBucket and is excluded from the stars rank, consistent with Table 5.8.

Software	**Q-Mean**	**Q-Rank**	**Stars**	**S-Rank**	**Δ**
Gazebo	10.0	1	1,337	17	+16
Drake	8.8	2	4,025	8	+6
MoveIt!	8.7	3	1,803	15	+12
OMPL	8.7	3	2,047	14	+11
Pinocchio	8.7	3	3,354	10	+7
ROS 2	8.7	3	5,475	5	+2
Habitat	8.0	7	3,662	9	+2
MuJoCo	7.9	8	13,440	4	−4
AirSim	7.8	9	18,159	1	−8
MRPT	7.8	9	2,133	13	+4
Webots	7.8	9	4,345	7	−2
CARLA	7.7	12	13,941	3	−9
OpenRAVE	7.7	12	801	21	+9
DART	7.6	14	1,081	19	+5
Bullet/ PyBullet	7.4	15	14,465	2	−13
CoppeliaSim	7.4	15	146	25	+10
Project Chrono	7.4	15	2,836	11	−4
YARP	7.3	18	590	22	+4
MORSE	7.2	19	367	23	+4
Simbody	7.0	20	2,521	12	−8
OpenRTM-aist	6.9	21	24	26	+5
PhysX (OSS)	6.8	22	4,532	6	−16
SOFA	6.8	22	1,194	18	−4
OROCOS	6.6	24	879	20	−4
ODE	6.3	25	—	—	—
Flightmare	6.1	26	1,353	16	−10
Newton Dynamics	5.6	27	24	26	−1
GraspIt!	5.0	28	213	24	−4

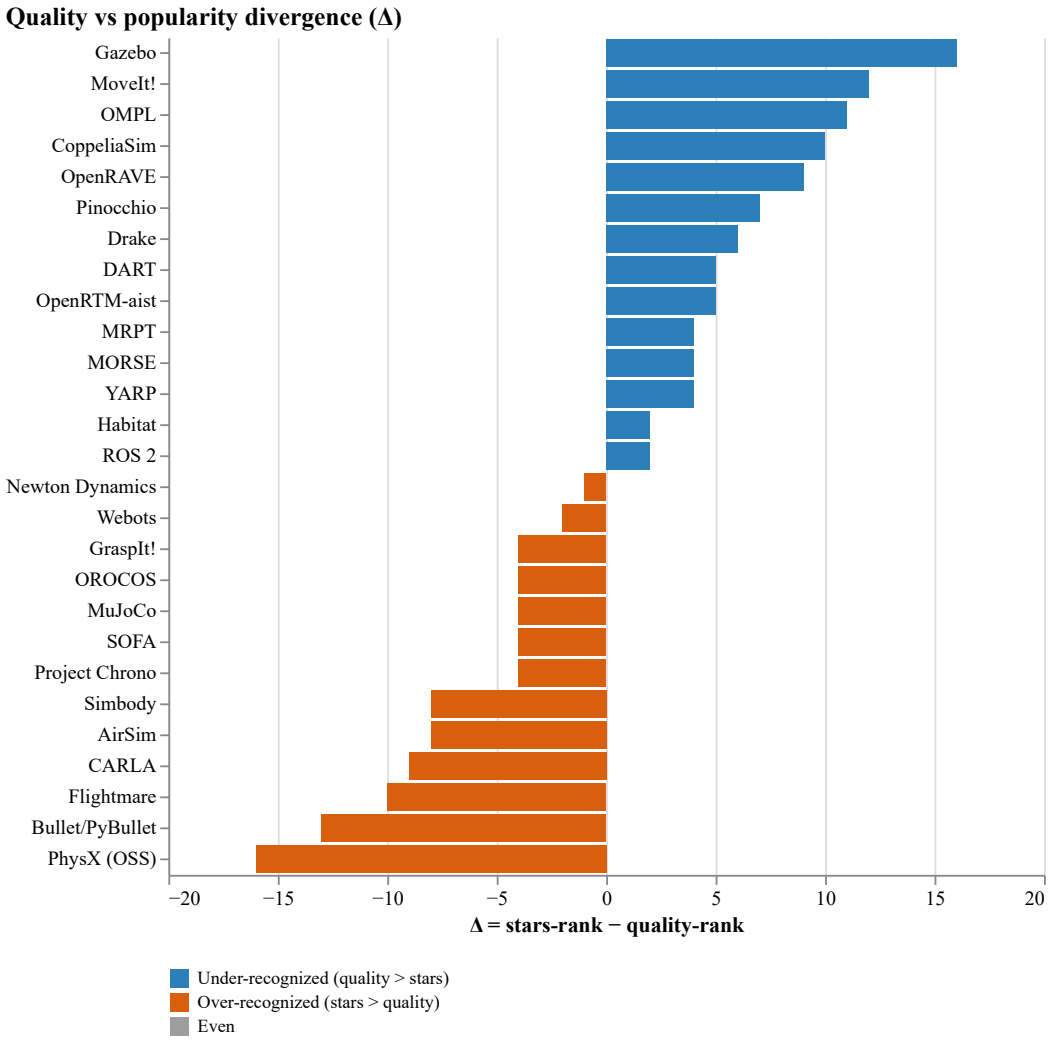

Figure 5.8: Rank divergence $\Delta = S - Q$ between GitHub-stars rank and quality-mean rank. Positive (blue), under-recognized for its quality; negative (orange), more popular than quality alone implies.

The 28 packages fall into four broad patterns, summarized visually in Figures 5.7 and 5.8.

Rough agreement at the top. Drake ($\Delta = +6$), ROS 2 ($\Delta = +2$), Pinocchio ($\Delta = +7$), and Habitat ($\Delta = +2$) appear in the top tier on both indicators; well-resourced projects with sustained development tend to combine strong engineering practice with substantial visibility.

High quality, low visibility. Gazebo ($\Delta = +16$), OMPL ($\Delta = +11$), and MoveIt! ($\Delta = +12$) score at or near the top on quality but sit well below their quality rank on stars. The Gazebo gap is explained in Section 5.11 (ROS-mediated usage undercounts stars). OMPL and MoveIt! are libraries embedded inside larger workflows and may attract attention as dependencies rather than as standalone projects.

High visibility, mid-pack quality. AirSim ($\Delta = -8$), Bullet/PyBullet ($\Delta = -13$), CARLA ($\Delta = -9$), and PhysX (OSS) ($\Delta = -16$) sit at or near the top on stars but in the middle or lower tier on the quality mean. These projects share characteristics: broad user bases drawn from adjacent fields (gaming, autonomous-driving research, real-time physics), corporate or game-engine origins, and uneven research-software quality practice.

Consistently low. GraspIt! ($\Delta = -4$) and Newton Dynamics ($\Delta = -1$) cluster near the bottom on both indicators; inactive or narrowly used projects tend to accumulate gaps in documentation, releases, and visibility together. For Newton Dynamics this placement reflects the repository-migration caveat (Section 5.1): against its original repository its popularity rank would sit mid-table, though its quality rank would not change. OpenRTM-aist sits near the bottom on both indicators as well, but with $\Delta = +5$ its quality rank (21) is modestly better than its stars rank (26), so

it belongs in this group only in the sense of low absolute standing on both axes, not as a case where popularity outruns quality.

These patterns are consistent with the medical imaging study's finding that GitHub popularity and measurement-based rankings do not align in a simple way (Smith et al., 2024a). They support the broader argument that treating stars as a quality proxy obscures meaningful differences across packages.

Chapter 6

Comparison with Other Research Software Domains

This chapter compares the findings for motion planning and simulation (MPS) software with prior studies of the state of the practice in other research software domains, especially Medical Imaging (MI) software (Smith et al., 2024a) and Lattice Boltzmann Method (LBM) software (Smith et al., 2024b). The goal is not to claim that one domain is better than another. The domains differ in age, user communities, computational goals, and measurement dates. Instead, the comparison identifies where MPS software follows patterns already observed in research software and where it appears to differ.

Following the LBM study (Smith et al., 2024b), the comparison uses three dimensions that map directly to RQ3, RQ4, and RQ5: repository artifacts, tool usage, and development principles, processes, and methodologies.

6.1 Comparison of Repository Artifacts

Table 6.1 summarizes the artifact observations from Chapter 5 (May 2026 snapshot).

Table 6.1: Summary of selected artifacts and visible development evidence in MPS packages.

Artifact or evidence type	**Packages**	**%**
Public source repository	28/28	100%
Installation instructions	28/28	100%
Installation automation	28/28	100%
Getting-started tutorial	28/28	100%
User manual	28/28	100%
API documentation	28/28	100%
Issue tracking	28/28	100%
Version control	28/28	100%
Unit tests visible or available	27/28	96%
Release notes	27/28	96%
Contribution or code-review guidance	26/28	93%
Explicit coding standard	14/28	50%
Documented expected user characteristics	1/28	4%

MPS shows near-universal user- and developer-facing infrastructure and compares well with prior studies. The MI study reported a user manual for 22 of 29 packages (Smith et al., 2024a), against all 28 in the current measurement. The LBM study found installation guides and tutorials common but API documentation rare (Smith et al., 2024b); every MPS package provides API documentation.

This difference likely reflects the way MPS software is used. Rather than standalone research scripts, many MPS projects are simulators, middleware, planning libraries, physics engines, or integration frameworks that users must install into larger toolchains, call through APIs, and combine with robotics ecosystems such as ROS. That use pattern creates pressure for installation documentation, tutorials, and API references. Their presence does not prove high quality, but it shows that the MPS ecosystem has adopted many of the artifacts that make research software inspectable and reusable.

The comparison is less uniformly positive for formal software engineering artifacts. MPS identifies an explicit coding standard in 14 of 28 packages, stronger than in the MI and LBM studies but still only half the sample; documented expected user characteristics, by contrast, appears in just one MPS package—rarer than in LBM, and not reported as a count by MI. The LBM study observed that research software projects often fall short of recommended artifacts such as roadmaps, coding standards, checklists, uninstall instructions, and formal design or requirements documentation (Smith et al., 2024b). The MPS results follow the same broad pattern: practical artifacts that help users install, run, and call the software are widespread, while artifacts that record design intent, expected user assumptions, and explicit development rules are less common.

Following the LBM (Smith et al., 2024b) and MI (Smith et al., 2024a) studies, which group repository artifacts into frequency bands, Table 6.2 classifies the MPS artifacts as *Common* (present in more than 67% of packages), *Uncommon* (33–67%), or *Rare* (fewer than 33%), following the band thresholds used in the MI study, and places each band beside the corresponding MI and LBM results.

Table 6.2: Repository artifacts grouped by frequency band—Common (>67% of packages), Uncommon (33–67%), and Rare (<33%)—for motion planning and simulation (MPS, this study), medical imaging (MI) (Smith et al., 2024a), and Lattice Boltzmann method (LBM) (Smith et al., 2024b) software. Percentages are taken over each study's own package count (MPS 28, MI 29, LBM 24). "n/r" marks an artifact the cited study does not report as a count.

Artifact	**MPS**	**MI**	**LBM**
Version control	Common (100%)	Common (100%)	Uncommon (67%)
Issue tracking	Common (100%)	Common (97%)	n/r
Installation instructions	Common (100%)	Uncommon (55%)	Common (100%)
Getting-started tutorial	Common (100%)	Uncommon (62%)	Common (79%)
User manual	Common (100%)	Common (76%)	Uncommon (54%)
API documentation	Common (100%)	Rare (24%)	Rare (0%)
Release notes	Common (96%)	Common (76%)	Uncommon (38%)
Contribution / review guidance	Common (93%)	Rare (28%)	n/r
Explicit coding standard	Uncommon (50%)	Rare (10%)	Rare (13%)
Documented user characteristics	Rare (4%)	n/r	Rare (21%)

The banding makes the cross-domain contrast explicit: almost every user- and developer-facing artifact is *Common* in the MPS sample, whereas several are only *Uncommon* or *Rare* in MI and LBM. Documented expected user characteristics is the scarcest artifact in the MPS sample and is *Rare* in LBM as well; the MI study does not report it as a count.

6.2 Comparison of Tool Usage

The LBM study discussed tool usage in terms of development tools, dependencies, and project management tools, with special attention to version control and continuous integration (Smith et al., 2024b). The MPS measurement provides strong evidence

for version control, issue tracking, build or installation automation, testing, API documentation, and visible development infrastructure.

Version control and issue tracking provide the most direct tool comparison. All 28 MPS packages use git-based version control on GitHub or an equivalent public platform, and all 28 use GitHub Issues or an equivalent tracker. The LBM study reported version control for 67% of all LBM packages and 73% of alive ones (Smith et al., 2024b). The MPS result is higher, partly because the inclusion rule requires public repository evidence and so filters out packages without suitable repositories (Chapter 3). Even so, the measured MPS ecosystem is strongly aligned with contemporary repository-centered development.

Build and installation automation are also widespread: all 28 packages provide build files, scripts, package-manager support, or container-based installation. MPS packages often depend on compiled C++, Python bindings, physics and graphics libraries, middleware, and OS packages that are hard to reproduce by hand, so this installability tooling (Section 5.2) reflects real maintainer investment.

Testing and verification tooling are also visible: unit tests for 27 of 28 packages, automated testing for 26, and assertions for 23. The LBM study treated test cases as a rare artifact (2 of 24 packages) and automated testing as a prerequisite for effective continuous integration (Smith et al., 2024b); in the MPS sample, testing evidence is far more common. This does not imply complete coverage or realistic robotics workloads, but it shows that test infrastructure is a visible part of most measured projects.

Continuous integration should be read more cautiously. Chapter 5 records CI or an equivalent setup for 26 of 28 packages and visible development-process evidence

(CI/CD workflows, contribution guides, pull request and issue templates) for 25 of 28. These counts show that MPS repositories commonly expose automation and workflow infrastructure; they do not reveal how deeply each project uses CI, and this report does not rank CI maturity separately.

API documentation should be distinguished from the tools that produce it. The MPS measurement found API documentation for all 28 packages, but documentation-generation tools such as Doxygen or Sphinx are recorded for only 12. The LBM paper reported that roughly half of LBM packages mentioned such tools and that API documentation itself was rare (Smith et al., 2024b).

Overall, MPS projects use modern repository and development tools consistently in the categories visible from public repositories.

6.3 Comparison of Principles, Processes, and Methodologies

Principles, processes, and methodologies are harder to assess from public artifacts alone, because process is partly social: repositories show issue trackers, pull requests, release notes, contribution guides, and workflows, but not how teams make design decisions, negotiate priorities, or conduct informal review outside the repository.

Within that limitation, MPS projects show more visible process structure than many research software projects in prior studies. Twenty-five of 28 packages show evidence of a defined development process (see Section 6.2), 27 of 28 publish release notes, and 26 of 28 provide contribution or code-review guidance. These indicate that many measured projects expect external users or contributors and have created at

least minimal structures for that interaction.

This contrasts with the LBM discussion, where process evidence was often informal and was strengthened by interviews: LBM developers described small-team practices, lead developer roles, informal peer review, and issue tracking (Smith et al., 2024b). The MPS interview component is reported separately (Chapter 7).

The two RQ3 gaps (Section 6.1) also limit the process picture: the missing coding standard and the single package documenting expected user characteristics weaken traceability between the intended audience and the design of tutorials, APIs, examples, and support channels.

Heterogeneity also affects the process comparison. A full simulator (Gazebo, Webots), a middleware platform (ROS 2), and a planning library (OMPL) do not share development needs: large ecosystem projects tend to expose more formal governance, release management, and contribution infrastructure, while smaller or older packages may offer source and documentation but little explicit process description. The commonality analysis (Chapter 4) explains why different process expectations may be reasonable for different kinds of MPS software, so the 28 packages should not be treated as interchangeable.

6.4 Summary of Cross-Domain Findings

Table 6.3 consolidates the MPS counts against the MI and LBM figures; the RQ3–RQ5 conclusions follow.

The consolidated counts confirm the pattern of Sections 6.1–6.3: MPS leads MI and LBM on practical, integration-oriented artifacts and visible tooling, while formal artifacts remain the shared gap. Chapter 10 answers RQ3–RQ5 individually.

Table 6.3: Cross-domain comparison of repository artifacts, tools, and process evidence: motion planning and simulation (MPS, this study) against the medical imaging (MI) and Lattice Boltzmann method (LBM) studies of the state of the practice. MPS counts are out of 28 packages (Chapter 5, May 2026); MI counts are out of 29 packages (Smith et al., 2024a) and LBM counts out of 24 packages (Smith et al., 2024b). Percentages are taken over each study's own package count, so the columns are comparable despite the different denominators.

Artifact / evidence	**MPS (n=28)**	**MI (n=29)**	**LBM (n=24)**
Repository artifacts (RQ3)			
Version control	28 (100%)	29 (100%)	16 (67%)
Issue tracking	28 (100%)	28 (97%)	n/r
Tutorial (getting-started)	28 (100%)	18 (62%)	19 (79%)
User manual	28 (100%)	22 (76%)	13 (54%)
Installation instructions	28 (100%)	16 (55%)	24 (100%)
API documentation	28 (100%)	7 (24%)	0 (0%)
Release notes	27 (96%)	22 (76%)	9 (38%)
Contribution / review guidance	26 (93%)	8 (28%)	n/r
Tools and process evidence (RQ4–RQ5)			
Unit tests available	27 (96%)	15 (52%)	2 (8%)
Continuous integration	26 (93%)	5 (17%)	3 (13%)
Documentation-generation tools	12 (43%)	n/r	13 (54%)
Explicit coding standard	14 (50%)	3 (10%)	3 (13%)
Documented user characteristics	1 (4%)	n/r	5 (21%)

Notes: MPS values restate Chapter 5. "n/r" marks an artifact that the cited study does not report as a count. The LBM API-documentation entry is zero because that study recorded no package with explicit API documentation (Smith et al., 2024b).

Chapter 7

Developer Interview Component

The public-artifact measurements in Chapters 4 through 6 answer RQ1–RQ5. RQ6–RQ9 require evidence from developers about pain points, practices, and cross-domain lessons that are not fully visible in public repositories. That interview evidence has not yet been collected or analyzed, so this chapter sets out the planned structure rather than empirical answers to RQ6–RQ9. Sections follow reporting flow rather than question order: the pain-point summary (RQ6) is followed by the practices addressing those pain points (RQ8), and the closing comparison treats RQ7 and RQ9 together.

The interview protocol, recruitment logic, ethics status, and analysis procedure are described in Section 3.9 of Chapter 3.

7.1 Interview Evidence Base

The evidence base will record the number of contacted projects, participating developers, represented MPS software packages, the form of the analysis records, and

any limits imposed by consent or confidentiality. Converting this chapter into a findings chapter requires a record of recruitment and consent, a coded set of developer responses, and a documented analysis linking pain points and practices back to the research questions.

The interview evidence is expected to complement the repository evidence used earlier in the report. Public artifacts show visible practices, such as documentation, issue tracking, releases, test evidence, and continuous-integration configuration. Interviews can provide less visible information, such as internal decision making, perceived development obstacles, informal review practices, funding or time constraints, and the reasons behind particular engineering choices.

7.2 Overview of Developer Pain Points

The RQ6 analysis will summarize recurring pain points reported by MPS developers, distinguishing domain-wide themes from issues specific to one package or software role.

Prior reports on the state of the practice (see the studies discussed in Chapter 6) suggest useful comparison categories, such as resource constraints, testing and correctness, documentation, maintainability, tooling, and project management, but the final categories for the MPS domain must reflect the interview responses.

7.3 Practices Used to Address Pain Points

RQ8 covers the specific best practices that MPS developers use, propose, or identify for addressing the RQ6 pain points and the RQ2 software qualities. The analysis

separates practices developers already use from those they propose, since implemented practice and aspiration are different kinds of evidence.

The analysis should connect each practice to the pain point it addresses. For example, if developers discuss practices related to documentation, testing, continuous integration, release management, contribution review, modularization, or user support, the chapter should state whether those practices were reported as current practice, past attempts, or future plans.

7.4 Analysis by Software Quality

The interview analysis will connect findings to the software qualities measured in Chapter 5, explaining why particular qualities are hard to improve or how teams try to improve them in practice.

Developer-reported evidence will be kept separate from public-artifact evidence; the differences should refine the interpretation of the measurements, not replace the measurement record.

7.5 Comparison with Other Research Software Domains

RQ7 and RQ9 compare MPS interview findings with prior state-of-the-practice studies, particularly the medical imaging and Lattice Boltzmann method reports discussed in Chapter 6, identifying pain points that recur across domains versus those specific to MPS software.

The comparison will also support future-practice recommendations. A practice observed in another research software domain should be presented as a candidate practice for MPS only when the connection to an MPS pain point is clear.

Chapter 8

Recommendations for Future Practice

The recommendations in this chapter come from the public-artifact measurements, the commonality analysis, and the comparison with prior studies of the state of the practice. They are not a ranking of packages or a prescription that every project adopt the same process. RQ8 and RQ9 ask which practices address developer pain points, but those pain points (RQ6) depend on the interview component (Chapter 7); the recommendations therefore rest on artifact evidence alone and should be revisited once interview evidence is available.

8.1 Recommendations by Software Quality

Read together, the measurements in Chapter 5 suggest a concrete, low-cost improvement for each measured quality area (surface reliability and robustness are treated together, since the same recommendation serves both). Where a recommendation is

developed further from the maintainer's point of view, it points to Section 8.3.

- **Installability.** All 28 packages give installation instructions, but many omit the supported operating-system and dependency versions; recording both, and providing at least one tested installation route, would remove the most common source of installation friction (Section 8.3).

- **Correctness and verifiability.** Test infrastructure is a strength—unit tests are visible for 27 of the 28 packages—so the useful addition is evidence a user can re-run: reference outputs, benchmark results, or worked examples published alongside the tests let users confirm correctness rather than infer it from the presence of tests.

- **Surface reliability and robustness.** The packages generally handle ordinary and malformed input in the surface checks, but that behaviour is rarely documented; a short statement of known limitations and expected failure modes would make the observed reliability something users can depend on.

- **Surface usability.** Getting-started tutorials and user manuals are universal, so the gap is audience definition: only one package documents its expected users, and a brief statement of intended audience and prerequisites would help users self-select and give tutorial authors a defined reader (Section 8.3).

- **Maintainability.** Release notes and contribution or code-review guidance are already present in most repositories (27 and 26 of 28); the few projects without them would most improve maintainability by adopting both, since these are the artifacts that let external contributors participate without direct contact with the original team.

- **Reusability.** Every package documents its API, but only 12 of 28 record a documentation-generation tool such as Doxygen or Sphinx; generating API documentation from the source, rather than maintaining it by hand, keeps it current as the interface evolves and lowers the cost of reuse.

- **Surface understandability.** All 28 packages format their code consistently, yet only half name an explicit coding standard; naming the standard, or publishing the formatter configuration, turns an implicit convention into a checkable artifact for new contributors (Section 8.3).

- **Visibility and transparency.** Continuous-integration evidence and a defined development process are visible for most packages, but a short note stating what the automation verifies—builds, tests, documentation, or releases—would make that evidence interpretable rather than merely present.

8.2 Recommendations for MPS Software Users

Users should choose packages by task fit before quality score. The selected packages serve different roles and are not direct substitutes, so a highly ranked simulator may be the wrong choice for a project that needs a planning library. The commonality analysis in Chapter 4 should therefore be read alongside the measurement results in Chapter 5.

Users should also treat public artifacts as evidence of adoption risk. Installation instructions, tutorials, API documentation, issue trackers, and release notes reduce the effort needed to evaluate and maintain a package; sparse documentation, weak release history, or inactive repositories raise it. Popularity indicators deserve caution:

Section 5.13 shows that GitHub stars and measured engineering quality diverge for several packages, so a star count is not a substitute for checking the artifacts directly.

8.3 Recommendations for MPS Software Maintainers

Maintainers should make development practices visible. The measured MPS projects often provide installation guides, tutorials, API references, and issue trackers, but formal and explanatory artifacts are less consistent. Coding standards, user assumptions, design rationale, and requirements-style information are useful because MPS packages are widely adopted by users and contributors who were not part of the original project.

The measurements quantify the three gaps flagged in Section 8.1: only one of the 28 packages documents its expected user characteristics, half (14 of 28) do not name a coding standard despite formatting consistently, and compatible operating-system versions are listed by only 61% of the packages and required dependency versions by 68%—the main source of the low installability scores in Section 5.2.

Maintainers should also keep repository identity legible. Several measured packages distribute development across repositories or have migrated repositories; Section 5.1 documents PhysX, OROCOS, and Newton Dynamics as examples. A prominent pointer from old locations to the current development home keeps community signals such as stars, forks, and issue history interpretable.

8.4 Recommendations for Future State-of-the-Practice Studies

Future studies of the state of the practice in heterogeneous software domains should separate domain membership from software role. Recording each package's role through commonality analysis helps avoid treating related packages as interchangeable products.

Future studies should also keep public-artifact evidence separate from developer-reported evidence. Repositories can reveal documentation, issue tracking, releases, testing evidence, and development visibility. They reveal less about internal decision making, informal review, funding constraints, and developer pain points. Those topics require interviews or other qualitative evidence, and claims about them should be tied to the interview records or consented summaries that support them.

Finally, future studies should record at measurement time the observations behind each impression score. Because the scores summarize more than the binary checklist answers (Section 3.7.1), they carry the subjectivity discussed in Chapter 9. A short per-package note on which observations drove each score would make the scores easier to audit and reproduce.

Chapter 9

Threats to Validity

Any study based on publicly available artifacts is constrained by what those artifacts can reveal. The threats below are organized by threat type: reliability, construct validity, internal validity, and external validity.

9.1 Reliability

All measurements are based on public evidence collected in May 2026 and reflect a snapshot of each repository at that moment (Section 2.7); the same project measured earlier or later might score differently. Some quality questions also require judgement, such as whether documentation is adequate or project metadata complete enough to support a score, and these were performed by a single assessor without an independent inter-rater check. Because one assessor measured all 28 packages in sequence, the criteria may also have been applied slightly differently as familiarity with the template grew—a learning or drift effect that a single rater cannot rule out, and that re-checking early packages against a fixed rubric would reduce but not eliminate. The rationale

is recorded in the measurement template (Section 3.7.1), but the impression scores summarize observations beyond the recorded checklist answers, so the recorded data alone cannot fully reconstruct them. The time budget of one to four hours per package bounds the depth of every check; the surface measures are shallow by design.

9.2 Construct Validity

The measurements use public artifacts as evidence for software qualities, a practical proxy that does not capture every aspect of the underlying constructs: visible testing evidence does not prove that a package is correct, and visible issue-tracking activity does not fully describe maintainability. Mention counts likewise measure how often a package appears across candidate sources, not how good the software is (Section 3.4). The choice of quality categories is itself a construct threat: the nine categories applied here (Section 3.7.1) are one reasonable decomposition of "software quality" rather than the only one, and because the impression scores involve assessor judgement (Section 9.1), a reader who defines or weights quality differently could reach different conclusions.

9.3 Internal Validity

The candidate set was built from academic survey papers, community-curated lists, and LLM-assisted discovery, with the LLM lists supplementing rather than driving the other two sources. Those lists reflect the training data of the model rather than an independent survey of the MPS domain, so a package unknown to the model could be absent from its output. The filtering decisions described in Section 3.5 further shape

the measured set, since a package can be excluded for being out of scope, closed source, inactive, or not measurable from public repositories.

The overall ranking also depends on how the per-quality scores are combined. The report uses an equal-weight aggregation (Chapter 5), which treats all nine qualities as equally important; a different, application-specific weighting could reorder the middle of the ranking. The sensitivity analysis reported in Chapter 5 perturbs the scores and re-computes the ranking, and finds the top and bottom stable, but the equal-weight choice itself remains a modelling decision rather than an empirical finding.

9.4 External Validity

The selected set covers open-source MPS software measurable through public artifacts. Proprietary platforms, discontinued projects, and narrowly scoped libraries may differ from the measured packages, so the results describe the selected open-source MPS sample, not all robotics software. Because the packages play different roles, comparisons are evidence about development practice across a domain rather than direct product-to-product comparisons.

The installation-based measurements were performed on a Linux virtual machine, so the installability, surface reliability, and surface robustness results reflect Linux behaviour only. The same packages may install and behave differently on Windows or macOS, and installation routes available at measurement time, such as distribution packages, may be absent on other operating systems or later distribution releases.

9.5 Threats to the Planned Interview Component

The developer-interview component (Chapter 7) is designed but not yet conducted, so the threats below will apply once it is carried out and should be read as limitations of that planned evidence rather than of the artifact measurements above. Participation is voluntary, so the developers who respond may not represent the domain: those who agree to be interviewed, or who maintain more visible projects, can differ systematically from those who do not, giving rise to self-selection and non-response bias. The number of interviews feasible within the project timeline is small, so the findings will illustrate rather than establish domain-wide patterns. Because the interviews are semi-structured and conducted by the researcher, the wording and follow-up questions can steer answers (interviewer bias), and developers may describe their practices more favourably than they are (self-report and social-desirability bias). These threats will be reduced, but not removed, by following the approved interview guide, separating reported current practice from aspiration, and reporting the number of participants and the packages they represent.

Chapter 10

Conclusions

10.1 Summary

This study assessed the state of the practice for robotics motion planning and simulation (MPS) software by adapting the Domain X methodology (Chapter 3) to a new domain. From 295 consolidated candidate names, 28 packages were selected and measured in May 2026 against the Domain X template, spanning full simulators, physics and dynamics engines, planning libraries, manipulation frameworks, and middleware. A commonality analysis (Chapter 4) described the selected packages as a software family, and the findings were compared with prior studies of the state of the practice for medical imaging and Lattice Boltzmann method software. A developer-interview component was designed and received MREB ethics clearance (project #8215, 2026-06-22); recruitment is underway, but the interviews have not yet been conducted, so their evidence is not reported here.

10.2 Answers to Research Questions

RQ1 (candidate packages). Filtering the 295 candidates by open-source availability, repository-based measurability, activity status, and scope coherence (Chapter 3, Appendix C) yielded the 28 measured packages; notable exclusions include MATLAB/Simulink, Unity, NVIDIA Isaac Sim, and RaiSim (closed core components), MRDS and USARSim (inactive), and GPMP2 and CHOMP (single-algorithm scope).

RQ2 (quality ranking). Across the nine quality categories (Chapter 5), the unweighted quality mean is led by Gazebo (10.0), Drake (8.8), and a tied group of MoveIt!, OMPL, Pinocchio, and ROS 2 (8.7); GraspIt! (5.0), Newton Dynamics (5.6), and Flightmare (6.1) rank lowest, consistent with their inactive, migrated, or low-activity status. Category means are tightly bunched (7.3–7.8), so the rankings separate the top and bottom tiers more clearly than the middle. An elicited-weight AHP ranking remains future work; an illustrative equal-weight AHP ranking reproduces the same leaders.

RQ3 (repository artifacts). Practical artifacts are near-universal in the MPS sample (Chapter 6): installation instructions, installation automation, tutorials, user manuals, API documentation, issue tracking, and version control are present for all 28 packages, which compares favourably with the medical imaging and LBM samples. The recurring gaps are formal artifacts: documented user characteristics (1 of 28) and explicit coding standards (14 of 28).

RQ4 (tool usage). The measured MPS projects are tightly connected to modern open-source tooling: git-based version control and issue tracking are universal,

continuous-integration evidence appears for 26 of 28 packages, and documentation-generation tools are recorded for 12. The repository-based inclusion rule partly explains this uniformity, but within the measured set tool adoption is stronger than in the LBM comparison point and at least comparable with medical imaging.

RQ5 (principles, processes, and methodologies). Process evidence is visible but artifact-bound: 25 of 28 packages expose a defined development process, 27 publish release notes, and 26 provide contribution guidance. What repositories do not reveal—design rationale, requirements assumptions, and internal decision making—remains under-documented in MPS software, the same broad pattern reported for other research software domains.

RQ6–RQ9 (pain points and practices). These require developer-interview evidence; Chapter 7 records the planned evidence base, and Chapter 8 gives artifact-based recommendations.

10.3 Key Findings

Four findings stand out from the artifact evidence. First, the practical engineering infrastructure of MPS software is strong: every measured package can be installed from documented instructions with some form of automation, and only two installations broke during measurement, both recoverably. Second, the weaknesses are concentrated in formal and explanatory artifacts rather than in tooling: user assumptions, coding standards, and design rationale are the recurring gaps. Third, community popularity and measured quality are different signals: Gazebo leads on quality yet sits mid-table on stars, and several highly starred packages place mid-table on quality, so popularity indicators are not quality proxies. Fourth, activity status only loosely

predicts measured quality: packages that go inactive decay at different rates depending on the engineering capital they accumulated while active, as the contrast between OpenRAVE and GraspIt! illustrates.

10.4 Future Work

The most immediate direction is the developer-interview component: completing recruitment, the interviews, and the analysis would answer RQ6–RQ9 and connect the measured artifact gaps to the pain points developers actually experience. A second direction is an elicited-weight AHP ranking, replacing the equal weights of Chapter 5 with weights obtained by pairwise comparison of the qualities. Longitudinal repeat measurements would track how the domain's practices change over time, and the same methodology could be extended to adjacent robotics software domains.

One further direction concerns the measurement process itself, since completing the template by hand is the most time-consuming part of the work. An exploratory trial in which an AI coding agent installed, verified, and measured two physics engines (Appendix F) produced factual entries tied to public sources but subjective scores that still needed human review. With a person checking the evidence, this kind of agent-assisted measurement could make it practical to cover more packages, or to repeat the measurements over time, without straining the time budget that constrains a manual study.

Appendix A

Measurement Template

This appendix reproduces the structure of the measurement template used in this report. The template is adapted from the template in Appendix B of Smith et al. (2021) and is grouped into project metadata, the nine quality categories described in Section 2.4, and repository metrics collected in the GitHub-style, GitStats-style, and SCC-style groups of Section 3.7.2. The full per-package responses are recorded in the measurement spreadsheet included in the supplementary project materials, with one row per question and one column per package. Each quality section ends with an overall impression score on a 1–10 scale.

Project Metadata

- Software name.
- URL.
- Affiliation: institution(s).

- Software purpose.
- Number of developers (commits-based count from repository logs).
- Funding source (unfunded, unclear, or funded with a string identifying the source).
- Initial release date.
- Last commit date.
- Status (alive, dead, or unclear; alive requires commits in the previous 18 months).
- License (GNU GPL, BSD, MIT, terms of use, trial, none, unclear, other).
- Supported platforms (Windows, Linux, macOS, Android, other).
- Software category (concept, public, or private).
- Development model (open source, freeware, commercial, or unclear).
- Number of publications mentioning the software.
- Source code URL.
- Programming languages.
- Whether performance was considered.

Installability

- Are installation instructions present?

- Are the instructions in one place (single document or web page)?
- Are the instructions linear (a single sequence of steps)?
- Are the instructions written assuming no dependencies are pre-installed?
- Are compatible operating system versions listed?
- Is the installation automated (makefile, script, installer, container)?
- If installation broke, was a descriptive error message displayed?
- Is there a specified way to validate the installation?
- Number of installation steps (including manual steps such as unzip).
- Operating system used for installation.
- Number of extra packages required before or during installation.
- Are required package versions listed?
- Are dependency installation instructions provided?
- Were there problems caused by uninstall (where applicable)?
- Overall installability impression (1–10).

Correctness and Verifiability

- Is there any reference to requirements specifications or theory manuals?

- What tools or techniques are used to build confidence in correctness (literate programming, automated testing, assertions, continuous integration, etc.)?
- Is there a getting-started tutorial?
- Are tutorial instructions linear?
- Does the tutorial provide an expected output?
- Does the tutorial output match the expected output?
- Are unit tests available?
- Is there evidence that performance was considered?
- Overall correctness and verifiability impression (1–10).

Surface Reliability

- Did the software break during installation?
- If installation broke, was a descriptive error message displayed?
- Did the software break during the initial tutorial test?
- If the tutorial test broke, was a descriptive error message displayed?
- If the tutorial test broke, was recovery possible?
- Overall surface reliability impression (1–10).

Surface Robustness

- Does the software handle unexpected or unanticipated input gracefully (data of the wrong type, malformed input, etc.)?
- For text input files, does the software handle modified line endings or other text-file edge cases?
- Overall surface robustness impression (1–10).

Surface Usability

- Is there a getting-started tutorial?
- Is there a user manual?
- Are expected user characteristics documented?
- What is the user support model (FAQ, user forum, email address for questions, etc.)?
- Overall surface usability impression (1–10).

Maintainability

- What is the current version number?
- Is there information on code review or contribution practices?
- Are non-code artifacts available, and which kinds?

- What issue tracking tool is in use?
- What percentage of identified issues are closed?
- What percentage of code consists of comments?
- Which version control system is in use?
- Overall maintainability impression (1–10).

Reusability

- How many code files are there?
- Is the API documented?
- Overall reusability impression (1–10).

Surface Understandability

- Is indentation and formatting style consistent?
- Is a coding standard explicitly identified?
- Are code identifiers consistent and meaningful?
- Are constants other than 0 and 1 hard-coded into the program?
- Are comments clear?
- Are parameters in the same order for all functions?

- Is the name or URL of any algorithm used mentioned?
- Is the code modularized?
- Overall surface understandability impression (1–10).

Visibility and Transparency

- Is the development process defined?
- Are there documents recording the development process and status?
- Is the development environment documented?
- Are release notes available?
- Overall visibility and transparency impression (1–10).

Repository Metrics

Counts collected from the primary repository at the measurement date. For projects with multiple repositories, the choice of the primary repository is recorded as part of the measurement notes.

- Number of text-based files.
- Number of binary files.
- Total lines in text-based files.
- Total lines added to text-based files.

- Total lines deleted from text-based files.
- Total number of commits.
- Commits per year for the last five years (or since project start if younger).
- Commits per month for the last twelve months.
- Code lines, comment lines, and blank lines in text-based files.
- Number of stars.
- Number of forks.
- Number of repository watchers.
- Number of open pull requests.
- Number of closed pull requests.
- Number of open issues.
- Number of closed issues.

Each substantive question above corresponds to one row in the measurement spreadsheet, with the per-package answer recorded in the column for that package. Including the supplementary entries (overall impression scores per category and the free-text additional-comments fields), the template contains approximately 108 fields per package.

Appendix B

Mention Count Source List

This appendix lists the sources used to construct the consolidated mention-count ranking described in Section 3.4. Sources fall into three groups: community-curated robotics software lists, academic survey or comparison papers, and auxiliary LLM-assisted discovery lists. The LLM-assisted lists were used only as supplementary discovery aids, as discussed in Section 3.3.3.

Community-Curated Sources

Table B.1 lists the 15 community-curated sources used for candidate discovery. Each source is a public list, GitHub Topic page, or Wikipedia article that aggregates robotics software relevant to motion planning or simulation. The four GitHub Topic pages are retained because they are recorded in the source catalogue in the supplementary project materials. They are dynamic GitHub aggregations, so copied page text was not preserved in the raw-material archive in the same way as the static curated-list pages.

Table B.1: Community-curated sources used for mention counting.

Source	**Description**
GitHub Topic: motion-planning	GitHub Topic page aggregating active motion planning projects.
GitHub Topic: robotics-simulation	GitHub Topic page aggregating robotics simulation projects.
GitHub Topic: path-planning	GitHub Topic page for path and motion planning.
GitHub Topic: robotics-navigation	GitHub Topic page for navigation and planning.
best-of-robot-simulators	Curated list of robotic simulators (knmcguire).
Awesome Robotics Libraries	Robotics library list with planning and simulation sections (jslee02).
Awesome Robotics (ahundt)	Comprehensive robotics list covering planning, optimization, and simulation.
Awesome Robotics (kiloreux)	Community-maintained robotics list with planning and simulation entries.
Awesome Motion Planning	Motion planning list covering classical and modern libraries (AGV-IIT-KGP).
Awesome Multibody Dynamics Simulation	Multibody dynamics and simulation list (jslee02).
Awesome Robotic Tooling	Robotic tools and libraries list (Ly0n).
Awesome ROS 2	ROS 2 ecosystem list including navigation, planning, and simulation (fkromer).
Awesome Webots	Webots-focused ecosystem list (lukicdarkoo).
Awesome Robotics Projects	Robotics projects list with planning and simulation components (mjyc).
Wikipedia: Robotics simulator	Wikipedia article aggregating robotics simulator entries.

Academic Sources

Table B.2 lists the ten academic survey and comparison papers used for candidate discovery, with full bibliographic entries in the reference list. These papers survey or compare simulators, physics engines, and development frameworks used in robotics research; the second column gives representative packages from the selected set that each paper discusses.

LLM-Assisted Discovery Sources

Table B.3 lists the auxiliary LLM-assisted discovery outputs archived with the source catalogue. These outputs were not treated as authoritative sources; they were used to surface additional candidate names that were then consolidated, filtered, and checked against public evidence before any package was selected for measurement.

LLM Query Details

Both LLM-assisted outputs were generated on 2 November 2025 by issuing the following identical prompt, independently, to two assistants: GPT-5, accessed through the ChatGPT web interface with web browsing enabled, and Gemini 2.5 Pro, accessed through the Gemini web interface. Recording the model, prompt, and query date makes this discovery step reproducible in principle. The prompt instructed the models not to draw on prior conversational memory, so that each list reflects a fresh query rather than accumulated context.

Based on authoritative sources, compile a list of approximately thirty robotics

Table B.2: Academic survey and comparison papers used for mention counting.

Academic source	**Example packages discussed**
Brugali et al. (2007), "Trends in Robot Software Domain Engineering" (book chapter)	ROS, OROCOS, OpenRTM-aist, Gazebo, YARP
Žlajpah (2008), "Simulation in Robotics"	GraspIt!, ODE, Newton Dynamics, CoppeliaSim
Reckhaus et al. (2010), "An Overview about Simulation and Emulation in Robotics"	GraspIt!, OpenRAVE, Gazebo
Harris and Conrad (2011), "Survey of Popular Robotics Simulators, Frameworks, and Toolkits"	Gazebo, ROS, OpenRAVE, MRPT, Webots
Staranowicz and Mariottini (2011), "A Survey and Comparison of Commercial and Open-Source Robotic Simulator Software"	Webots, Gazebo
Iñigo-Blasco et al. (2012), "Robotics Software Frameworks for Multi-Agent Robotic Systems Development"	ROS, OROCOS, OpenRTM-aist, Gazebo, YARP
Collins et al. (2021), "A Review of Physics Simulators for Robotic Applications"	Gazebo, CoppeliaSim, Webots, MuJoCo, CARLA, SOFA, Project Chrono
Körber et al. (2021), "Comparing Popular Simulation Environments in the Scope of Robotics and Reinforcement Learning"	Gazebo, MuJoCo, PyBullet, Webots, DART, Simbody, PhysX
Farley et al. (2022), "How to Pick a Mobile Robot Simulator"	Gazebo, Webots, MORSE, CoppeliaSim, ODE, DART, Simbody
Kargar et al. (2024), "Emerging Trends in Realistic Robotic Simulations"	Gazebo, Webots, CoppeliaSim, MuJoCo, MORSE, PhysX, ODE, DART

Table B.3: LLM-assisted discovery sources archived with the source catalogue.

Source	**Role in candidate discovery**
GPT-generated robotics software list	Auxiliary broad-discovery list of open-source robotics software, simulators, planning libraries, physics engines, and related tooling.
Gemini-generated robotics software table	Auxiliary consolidated table of robotics frameworks, motion-planning tools, physics engines, and simulation platforms.

software packages related to motion planning, control, and simulation. Include both general-purpose frameworks and specialized toolkits, indicating their origin, primary domain focus, and level of usage or citation popularity. Do not use any prior conversational memory.

The source catalogue with community-source URLs and archived LLM-assisted outputs, the raw captured text from the static public web lists, and the consolidated ranking with per-package mention counts are all preserved in the supplementary project materials that accompany this report.

Appendix C

Full Candidate Software List

This appendix records the leading candidate software packages considered during candidate discovery (Section 3.3). Consolidating the names collected from academic survey papers, community-curated lists, and LLM-assisted discovery produced 295 unique candidates; the full ranked list is preserved in the supplementary project materials, and the counted sources are listed in Appendix B. Table C.1 records every candidate that received at least four mentions, together with the explicitly excluded tools and a representative sample of lower-count candidates.

Each entry shows the candidate's primary role in the MPS workflow, its consolidated mention count, and its status after filtering. The status values are defined as follows. *Measured*: selected for measurement and reported in Chapter 5. *Excluded*: removed by an explicit, documented filtering decision (open-source availability, activity, or scope), with the rationale for each exclusion given in Appendix D. *Not selected*: remained in the candidate pool but was not chosen, because it fell below the mention-count cutoff, failed one of the filtering criteria of Section 3.5, or is subsumed by a measured package. In the Mentions column, "w/ X" means the candidate is counted

within the consolidated entry of measured package X, and "—" means the candidate has no entry in the consolidated mention-count list and entered the candidate discussion through the scope and filtering review rather than through the counted sources.

Table C.1: Candidate software packages considered for measurement, with consolidated mention counts.

Package	**Primary role**	**Mentions**	**Status**
AirSim	Simulator (autonomous vehicles)	7	Measured
Bullet/PyBullet	Physics engine	16	Measured
CARLA	Simulator (autonomous driving)	6	Measured
CoppeliaSim (V-REP)	Simulator (general robotics)	13	Measured
DART	Dynamics library	6	Measured
Drake	Toolbox (dynamics, planning, control)	6	Measured
Flightmare	Simulator (quadrotor)	4	Measured
Gazebo	Simulator (general robotics)	22	Measured
GraspIt!	Manipulation (grasping)	4	Measured
Habitat	Simulator (embodied AI)	4	Measured
MORSE	Simulator (academic robotics)	6	Measured
MoveIt!	Manipulation (motion planning)	4	Measured
MRPT	Library (mobile robotics)	4	Measured
MuJoCo	Physics engine	12	Measured
Newton Dynamics	Physics engine	4	Measured
ODE (Open Dynamics Engine)	Physics engine	10	Measured
OMPL	Motion planning library	5	Measured
OpenRAVE	Planning framework	9	Measured
OpenRTM-aist	Middleware (RT components)	4	Measured

Package	Primary role	Mentions	Status
OROCOS	Control framework	5	Measured
PhysX (open-source SDK)	Physics engine	8	Measured
Pinocchio	Dynamics library	5	Measured
Project Chrono	Multi-physics simulation	4	Measured
ROS 2	Middleware and SDK	10	Measured
Simbody	Multibody dynamics library	5	Measured
SOFA	Simulation framework	5	Measured
Webots	Simulator (general robotics)	16	Measured
YARP	Middleware (humanoid robotics)	4	Measured
NVIDIA Isaac Sim	Simulator (high-fidelity, AI)	9	Excluded (non-OSS)
Unity	Game engine / simulator	8	Excluded (non-OSS)
MATLAB / Simulink	Robotics development environment	6	Excluded (non-OSS)
RaiSim	Physics simulator	4	Excluded (non-OSS)
MRDS	Simulator (Microsoft)	5	Excluded (inactive)
USARSim	Simulator (robot/automation)	5	Excluded (inactive)
GPMP2	Motion planning algorithm (reference impl.)	—	Excluded (scope)
CHOMP	Motion planning algorithm (reference impl.)	1	Excluded (scope)
Player / Stage	Middleware and 2D/3D simulation	6	Not selected
ROS (ROS 1)	Middleware (predecessor of ROS 2)	w/ ROS 2	Not selected
iGibson / OmniGibson	Simulator (interactive household)	3	Not selected
ManiSkill	Simulator (manipulation benchmark)	3	Not selected

Package	Primary role	Mentions	Status
SAPIEN	Simulator (general robotics)	3	Not selected
RBDL	Rigid body dynamics library	3	Not selected
Crocoddyl	Optimal control library (DDP)	3	Not selected
RobotStudio	Industrial robot simulator (ABB)	3	Not selected
robosuite	Simulator (manipulation, MuJoCo-based)	w/ MuJoCo	Not selected
Orocos KDL	Kinematics and dynamics (component of OROCOS)	w/ OROCOS	Not selected
TrajOpt	Trajectory optimization library	2	Not selected
SBPL	Motion planning library (search-based)	2	Not selected
KineoWorks	Path planning (Siemens, commercial)	2	Not selected
Robotics Toolbox for MATLAB	Robotics toolbox (MATLAB)	2	Not selected
FCL	Collision and proximity queries	2	Not selected
Crazyswarm / Crazyswarm2	Multi-drone control and planning	2	Not selected
CasADi	AD and nonlinear optimization	2	Not selected
RoboDK	Industrial robot simulator and OLP	2	Not selected
Tesseract	Planning environment (optimization)	1	Not selected
iDynTree	Multibody dynamics, estimation	1	Not selected
STOMP	Motion planning algorithm	1	Not selected
Navigation2 (Nav2)	Mobile robot navigation stack	1	Not selected
Polymetis	Robot controllers (PyTorch)	1	Not selected

Package	Primary role	Mentions	Status
acados	Real-time NMPC and MHE solvers	1	Not selected
do-mpc	Python MPC and MHE	1	Not selected
Delmia	Digital manufacturing suite (Dassault)	1	Not selected
ros2_control	Robot control framework (ROS 2)	—	Not selected

The 28 measured packages were selected after the mention-count ranking and the filtering criteria of Section 3.5. Among the not-selected entries, proprietary or industrial tools such as *KineoWorks*, *Robotics Toolbox for MATLAB*, *RoboDK*, *RobotStudio*, and *Delmia* fall outside the open-source criterion; most others fell below the mention-count cutoff. *Player / Stage*, despite six mentions, is both inactive and largely superseded by actively maintained successors such as Gazebo (which originated in the same Player Project), so it was not selected; this is distinct from the inactive but still-prominent packages, such as MORSE, that were retained and measured (Section 3.5.3). *ROS* (ROS 1) is consolidated with ROS 2 in the mention counting; the active successor was measured so that the results reflect current practice.

Appendix D

Excluded Software Packages

This appendix records software packages considered during candidate discovery but excluded before measurement, following the criteria in Section 3.5. A small number were also excluded to keep the selected set methodologically coherent.

Excluded for Lack of Open-Source Availability

The packages in Table D.1 have proprietary core components or closed source code, which prevents the repository-based measurement described in Section 3.7. Dr. Giamou confirmed that excluding these tools on methodological grounds does not misrepresent the MPS domain.

Table D.1: Packages excluded for lack of open-source availability.

Package	**Reason for exclusion**
MATLAB / Simulink	Proprietary software; source code not publicly available; development process not accessible through public repositories.
Unity	Game engine with robotics simulation capabilities, but the core simulation engine is proprietary.
NVIDIA Isaac Sim	Built on NVIDIA Omniverse; core platform components are not open source.
RaiSim	High-performance physics simulator; source code not publicly available under an open-source license.

Excluded for Inactivity

The packages in Table D.2 were excluded under the Domain X alive/dead rule (Section 3.5.3): no commits or releases within the 18 months preceding the May 2026 measurement date, and no current maintenance.

Table D.2: Packages excluded on activity grounds.

Package	**Reason for exclusion**
MRDS (Microsoft Robotics Developer Studio)	No updates since 2012; no longer supported by Microsoft.
USARSim (Unified System for Automation and Robot Simulation)	No active maintenance; does not represent current practice.

Excluded to Maintain Methodological Coherence

The packages in Table D.3 are public reference implementations of single motion planning algorithms rather than general-purpose simulators, planning frameworks, physics engines, or middleware. Comparing them alongside full stacks such as Gazebo, MoveIt!, or ROS 2 would distort both the commonality analysis and the cross-package measurements. They were excluded for methodological consistency, not on quality grounds.

Table D.3: Packages excluded to maintain methodological coherence.

Package	**Reason for exclusion**
GPMP2 (Gaussian Process Motion Planner 2)	Single-algorithm optimization-based motion planning reference implementation; narrower in scope than the general-purpose packages that dominate the candidate list.
CHOMP (Covariant Hamiltonian Optimization for Motion Planning)	Single-algorithm optimization-based motion planning reference implementation; narrower in scope than the general-purpose packages that dominate the candidate list.

The full filtered candidate list is preserved in the supplementary project materials; Appendix C records the leading candidates with their mention counts and dispositions.

Appendix E

Interview Materials

This appendix reproduces the semi-structured interview guide referenced in Section 3.9. The guide contains 18 questions in three groups: the interviewee's background, the development process and tools, and the software's users. The interviewer may adjust the exact wording during the conversation and may use short clarification prompts to ask for more detail or concrete examples.

Interviewee background.

1. What is your current position or title? What degrees do you hold?
2. What is your contribution to, or relationship with, the software?
3. How long have you been involved with this software?

Developers, development process, tools, and techniques.

4. How large is the development group?
5. What is the typical background of a developer?

6. Currently, what are the most significant pain points in your development process? Specifically, what are the pain points in requirements definition, design, implementation, and verification?

7. How does documentation fit into your development process? Would improved documentation help with the obstacles you typically face?

8. In the past, was there any major obstacle to your development process that has been solved? How did you solve it?

9. What is your software development model? For example, waterfall, agile, or another model.

10. What is your project management process? Are any project management tools used?

11. Is it hard to ensure the correctness of the software? If there were obstacles, what methods have been considered or practiced to improve the situation? If practiced, did they work?

12. When designing the software, did you consider the ease of future changes? For example, will it be hard to change the system structure, modules, or code blocks? What measures have been taken to support future changes and maintenance?

13. Do you have any concern that your computational results will not be reproducible in the future? Have you taken any steps to ensure reproducibility?

14. What positive contributions do LLM-based tools have on your development process with respect to documentation, coding, and testing?

15. What negatives are associated with LLM-based tools?

Users.

16. What is the typical background of a user?

17. Can you provide instances where users have misunderstood or misused the software? What actions, if any, were taken to address usability issues?

18. Do you think the current documentation clearly conveys all necessary knowledge to users? If yes, how did you successfully achieve it? If no, what improvements are needed?

Appendix F

Exploratory AI-Assisted Measurement

This appendix records a small exploratory trial in which an AI coding agent carried out this study's measurement protocol on its own, over two physics engines from the selected set (PyBullet and MuJoCo), using the template of Appendix A and the procedure of Section 3.7. It is not part of the validated results in Chapter 5; it is a feasibility check and the basis for the future-work direction in Section 10.4. The agent worked in a Linux shell with file-system and network access in early June 2026, so its repository snapshots fall slightly later than the May 2026 snapshot of the main study.

Task Given to the Agent

The agent was given a single natural-language prompt and no installation commands; it had to locate the official documentation, install each package, verify the installation,

and complete the template unaided. The prompt is reproduced below.

> Read the CSV file in the current directory and use the first column ("Metric") as the measurement template. Measure two software packages, PyBullet and MuJoCo. The requirements were:
>
> - Only fill in the columns for these two packages; do not modify the other software columns. A new CSV file may be created.
> - Do not rely on user-provided installation commands; find the official documentation, GitHub README, or project website to determine the installation and verification steps.
> - Follow each metric in the CSV to complete installation, verification, observation, and data entry.
> - Take a screenshot at every step and save it to a screenshots folder. Screenshots must cover at least: reading the CSV, finding the official documentation, the installation steps, the verification steps, error messages or successful output, and the final CSV result.
> - Evidence may come only from official documentation, GitHub, terminal output, and commands actually executed.
> - Write "unclear" for uncertain values and "n/a" for values that do not apply.
> - Also write a `notes.md` file recording the selected packages, the official links used, the commands run at each step, the screenshot filenames, whether installation succeeded, whether verification succeeded, which CSV cells were filled or changed, and any problems encountered with their recovery.
>
> Complete Bullet/PyBullet first, then MuJoCo.

Official Sources Located by the Agent

The agent found the official sources in Table F.1 and used them to choose the installation and verification steps.

Table F.1: Official sources the agent used for the two packages.

Package	**Resource**	**Location**
PyBullet	Website and forum	`https://pybullet.org`
	Source repository	`https://github.com/bulletphysics/bullet3`
	Python package	`https://pypi.org/project/pybullet/`
MuJoCo	Website	`https://mujoco.org`
	Source repository	`https://github.com/google-deepmind/mujoco`
	Documentation	`https://mujoco.readthedocs.io`

Installation and Verification

Both packages installed with a single `pip` command inside a Python virtual environment, and both were verified with a short program that the agent wrote and ran. Table F.2 summarizes the outcome.

Table F.2: Installation and verification outcome of the AI-assisted trial.

Item	**PyBullet**	**MuJoCo**
Install command	`pip install pybullet`	`pip install mujoco`
Installed version	3.2.7	3.9.0
Installation steps	1 (single pip command)	1 (single pip command)
Extra packages	none (self-contained)	bundled with the wheel
Operating system	Linux (Ubuntu)	Linux (Ubuntu)
Verification	Loaded the bundled `plane.urdf` model in DIRECT mode	Compiled an inline XML model of a free-jointed box on a plane
Result	Succeeded	Succeeded

The agent also queried the GitHub API and recorded the template's project metadata, each entry tied to the source it read. For PyBullet (`bulletphysics/bullet3`) it recorded 311 contributors, a zlib license, a first commit in April 2011, and a most recent push in October 2025; for MuJoCo (`google-deepmind/mujoco`), 116 contributors, an Apache-2.0 license, a first commit in August 2021, and a most recent push in June 2026. These contributor counts come from the GitHub contributors API, which returns fewer entries than the commit-log method of the main study (Section 3.7.2; 333 developers for Bullet/PyBullet, 138 for MuJoCo), so the two are not directly comparable.

Problems Encountered and Recovery

The agent encountered three problems and recovered from each without manual intervention. First, a direct `pip install` failed because the system Python was externally managed; the agent created a virtual environment and installed into it. Second, its first PyBullet verification script queried the physics engine before connecting to it; the agent reordered the calls so the connection came first. Third, no screenshot utility was present, so the agent installed one.

Limitations and Relationship to the Main Study

As a single run over two packages by one agent, this shows feasibility, not a result. The checkable, factual entries (installation command, installed version, license, contributor count, verification result) were each tied to a public source and straightforward to confirm. The subjective impression scores and free-text judgements were not validated here, and in places disagree with the agent's own notes, which is the clearest reason a human must still review the output before it can serve as evidence.

The trial's artifacts, the completed template and the notes document, are preserved in the supplementary project materials. The trial motivates the agent-assisted measurement under human supervision discussed in Section 10.4.

Bibliography

Richard J. Acton. Research software sharing, publication, & distribution checklists. Webinar, Best Practices for HPC Software Developers (HPC-BP) series, IDEAS Productivity, May 2026.

Noriaki Ando, Takashi Suehiro, Kosei Kitagaki, Tetsuo Kotoku, and Woo-Keun Yoon. RT-middleware: Distributed component middleware for RT (robot technology). In *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3933–3938, 2005. doi: 10.1109/IROS.2005.1545521.

Davide Brugali, Arvin Agah, Bruce MacDonald, Issa A. D. Nesnas, and William D. Smart. Trends in robot software domain engineering. In *Software Engineering for Experimental Robotics*, volume 30 of *Springer Tracts in Advanced Robotics*. Springer, Berlin, Heidelberg, 2007. doi: 10.1007/978-3-540-68951-5_1.

Herman Bruyninckx. Open robot control software: The OROCOS project. In *Proceedings of the 2001 IEEE International Conference on Robotics and Automation (ICRA)*, volume 3, pages 2523–2528. IEEE, 2001. doi: 10.1109/ROBOT.2001.933002.

Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent

Lamiraux, Olivier Stasse, and Nicolas Mansard. The Pinocchio C++ library – a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *IEEE/SICE International Symposium on System Integration (SII)*, pages 614–619, Paris, France, 2019. IEEE. doi: 10.1109/SII.2019.8700380.

Howie Choset, Kevin M. Lynch, Seth Hutchinson, George Kantor, Wolfram Burgard, Lydia E. Kavraki, and Sebastian Thrun. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, Cambridge, MA, 2005.

David Coleman, Ioan A. Şucan, Sachin Chitta, and Nikolaus Correll. Reducing the barrier to entry of complex robotic software: A MoveIt! case study. *Journal of Software Engineering for Robotics*, 5(1):3–16, 2014. doi: 10.6092/JOSER_2014_05_01_p3.

Jack Collins, Shelvin Chand, Anthony Vanderkop, and David Howard. A review of physics simulators for robotic applications. *IEEE Access*, 9:51416–51431, 2021. doi: 10.1109/ACCESS.2021.3068769.

Erwin Coumans and Yunfei Bai. PyBullet, a python module for physics simulation for games, robotics and machine learning. `http://pybullet.org`, 2016–2021.

Rosen Diankov and James Kuffner. OpenRAVE: A planning architecture for autonomous robotics. Technical Report CMU-RI-TR-08-34, Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, July 2008.

Alexey Dosovitskiy, Germán Ros, Felipe Codevilla, Antonio López, and Vladlen

Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, volume 78 of *Proceedings of Machine Learning Research*, pages 1–16. PMLR, 2017.

Gilberto Echeverria, Nicolas Lassabe, Arnaud Degroote, and Séverin Lemaignan. Modular open robots simulation engine: MORSE. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, pages 46–51. IEEE, 2011. doi: 10.1109/ICRA.2011.5980252.

Andrew Farley, Jie Wang, and Joshua A. Marshall. How to pick a mobile robot simulator: A quantitative comparison of CoppeliaSim, Gazebo, MORSE and Webots with a focus on accuracy of motion. *Simulation Modelling Practice and Theory*, 117:102629, 2022. doi: 10.1016/j.simpat.2022.102629.

François Faure, Christian Duriez, Hervé Delingette, Jérémie Allard, Benjamin Gilles, Stéphanie Marchesseau, Hugo Talbot, Hadrien Courtecuisse, Guillaume Bousquet, Igor Peterlik, and Stéphane Cotin. SOFA: A multi-model framework for interactive physical simulation. In *Soft Tissue Biomechanical Modeling for Computer Assisted Surgery*, volume 11 of *Studies in Mechanobiology, Tissue Engineering and Biomaterials*, pages 283–321. Springer, Berlin, Heidelberg, 2012. doi: 10.1007/8415_2012_125.

Jo Erskine Hannay, Carolyn MacLeod, Janice Singer, Hans Petter Langtangen, Dietmar Pfahl, and Greg Wilson. How do scientists develop and use scientific software? In *2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, pages 1–8, 2009. doi: 10.1109/SECSE.2009.5069155.

Adam Harris and James M. Conrad. Survey of popular robotics simulators, frameworks, and toolkits. In *Proceedings of IEEE SoutheastCon 2011*, pages 243–249, Nashville, TN, USA, 2011. doi: 10.1109/SECON.2011.5752942.

Pablo Iñigo-Blasco, Fernando Diaz-del Rio, Ma Carmen Romero-Ternero, Daniel Cagigas-Muñiz, and Saturnino Vicente-Diaz. Robotics software frameworks for multi-agent robotic systems development. *Robotics and Autonomous Systems*, 60 (6):803–821, 2012. doi: 10.1016/j.robot.2012.02.004.

Seyed Mohamad Kargar, Borislav Yordanov, Carlo Harvey, and Ali Asadipour. Emerging trends in realistic robotic simulations: A comprehensive systematic literature review. *IEEE Access*, 12, 2024.

Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154, 2004. doi: 10.1109/IROS.2004.1389727.

Marian Körber, Johann Lange, Stephan Rediske, Simon Steinmann, and Roland Glück. Comparing popular simulation environments in the scope of robotics and reinforcement learning, 2021.

Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, New York, NY, USA, 2006.

Jeongseok Lee, Michael X. Grey, Sehoon Ha, Tobias Kunz, Sumit Jain, Yuting Ye,

Siddhartha S. Srinivasa, Mike Stilman, and C. Karen Liu. DART: Dynamic animation and robotics toolkit. *Journal of Open Source Software*, 3(22):500, 2018. doi: 10.21105/joss.00500.

Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. doi: 10.1126/scirobotics.abm6074.

Giorgio Metta, Paul Fitzpatrick, and Lorenzo Natale. YARP: Yet another robot platform. *International Journal of Advanced Robotic Systems*, 3(1):43–48, 2006. doi: 10.5772/5761.

Peter Michalski. State of the practice for lattice boltzmann method software. M.eng. thesis, McMaster University, 2021.

Olivier Michel. Cyberbotics ltd. Webots: Professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):39–42, 2004. doi: 10.5772/5618.

Andrew T. Miller and Peter K. Allen. GraspIt! a versatile simulator for robotic grasping. *IEEE Robotics & Automation Magazine*, 11(4):110–122, 2004. doi: 10.1109/MRA.2004.1371616.

David Lorge Parnas. On the design and development of program families. *IEEE Transactions on Software Engineering*, SE-2(1):1–9, 1976.

Prakash Prabhu, Thomas B. Jablin, Arun Raman, Yun Zhang, Jialu Huang, Hanjun Kim, Nick P. Johnson, Feng Liu, Soumyadeep Ghosh, Stephen Beard, Taewook Oh, Matthew Zoufaly, David Walker, and David I. August. A survey of the practice of computational science. In *Proceedings of 2011 International Conference for High*

Performance Computing, Networking, Storage and Analysis (SC '11), pages 1–12, New York, NY, USA, 2011. ACM. doi: 10.1145/2063348.2063374.

Michael Reckhaus, Nico Hochgeschwender, Jan Paulus, Azamat Shakhimardanov, and Gerhard K. Kraetzschmar. An overview about simulation and emulation in robotics. In *Proceedings of the Workshop on Simulation Technologies in the Robot Development Process, International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAR 2010)*, Darmstadt, Germany, 2010.

Eric Rohmer, Surya P. N. Singh, and Marc Freese. V-REP: A versatile and scalable robot simulation framework. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1321–1326, 2013. doi: 10.1109/IROS.2013.6696520.

Thomas L. Saaty. *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. McGraw-Hill Publishing Company, New York, New York, 1980.

Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A platform for embodied AI research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9339–9347, 2019. doi: 10.1109/ICCV.2019.00943.

Shital Shah, Debadeepta Dey, Chris Lovett, and Ashish Kapoor. AirSim: High-fidelity visual and physical simulation for autonomous vehicles. In Marco Hutter and Roland Siegwart, editors, *Field and Service Robotics: Results of the 11th International Conference*, volume 5 of *Springer Proceedings in Advanced Robotics*, pages 621–635. Springer, 2018. doi: 10.1007/978-3-319-67361-5_40.

Michael A. Sherman, Ajay Seth, and Scott L. Delp. Simbody: Multibody dynamics for biomedical research. *Procedia IUTAM*, 2:241–261, 2011. doi: 10.1016/j.piutam.2011.04.023.

W. Spencer Smith, D. Adam Lazzarato, and Jacques Carette. State of the practice for mesh generation and mesh processing software. *Advances in Engineering Software*, 100:53–71, 2016.

W. Spencer Smith, D. Adam Lazzarato, and Jacques Carette. State of the practice for GIS software, 2018a.

W. Spencer Smith, Yue Sun, and Jacques Carette. Statistical software for psychology: Comparing development practices between CRAN and other communities, 2018b.

W. Spencer Smith, Zheng Zeng, and Jacques Carette. Seismology software: State of the practice. *Journal of Seismology*, 22:755–788, 2018c. doi: 10.1007/s10950-018-9731-3.

W. Spencer Smith, Jacques Carette, Oluwaseun Owojaiye, Peter Michalski, and Ao Dong. Methodology for assessing the state of the practice for domain X, 2021.

W. Spencer Smith, Ao Dong, Jacques Carette, and Michael D. Noseworthy. State of the practice for medical imaging software, 2024a.

W. Spencer Smith, Peter Michalski, Jacques Carette, and Zahra Keshavarz-Motamed. State of the practice for lattice boltzmann method software. *Archives of Computational Methods in Engineering*, 31:313–350, 2024b. doi: 10.1007/s11831-023-09981-2.

Yunlong Song, Selim Naji, Elia Kaufmann, Antonio Loquercio, and Davide Scaramuzza. Flightmare: A flexible quadrotor simulator. In *Proceedings of the 2020 Conference on Robot Learning (CoRL)*, volume 155 of *Proceedings of Machine Learning Research*, pages 1147–1157. PMLR, 2021.

Aaron Staranowicz and Gian Luca Mariottini. A survey and comparison of commercial and open-source robotic simulator software. In *Proceedings of the 4th International Conference on PErvasive Technologies Related to Assistive Environments (PETRA '11)*, pages 1–8. ACM, 2011. doi: 10.1145/2141622.2141689.

Ioan A. Şucan, Mark Moll, and Lydia E. Kavraki. The Open Motion Planning Library. *IEEE Robotics & Automation Magazine*, 19(4):72–82, 2012. doi: 10.1109/MRA.2012.2205651.

Alessandro Tasora, Radu Serban, Hammad Mazhar, Arman Pazouki, Daniel Melanz, Jonathan Fleischmann, Michael Taylor, Hiroyuki Sugiyama, and Dan Negrut. Chrono: An open source multi-physics dynamics engine. In *High Performance Computing in Science and Engineering (HPCSE 2015)*, volume 9611 of *Lecture Notes in Computer Science*, pages 19–49. Springer, 2016. doi: 10.1007/978-3-319-40361-8_2.

Russ Tedrake and the Drake Development Team. Drake: Model-based design and verification for robotics, 2019. URL `https://drake.mit.edu`.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.

David M. Weiss. Commonality analysis: A systematic process for defining families. In *Development and Evolution of Software Architectures for Product Families (ARES '98)*, volume 1429 of *Lecture Notes in Computer Science*, pages 214–222. Springer, Berlin, Heidelberg, 1998. doi: 10.1007/3-540-68383-6_30.

Leon Žlajpah. Simulation in robotics. *Mathematics and Computers in Simulation*, 79 (4):879–897, 2008.